

NLP-1: Interview Guide to Core NLP and Language Models

This guide covers the requested NLP topics from basics to interview depth. Each topic follows the same placement-friendly structure: overview, intuition, core ideas, working process, math, code, evaluation, mistakes, limitations, variations, interview questions, practice, projects, revision, and cheat sheet.

Text Preprocessing

1. Overview

Text preprocessing converts raw text into a cleaner, more consistent form before feeding it into an NLP model. Raw text contains noise: casing differences, punctuation, HTML tags, emojis, URLs, spelling variants, extra spaces, and domain-specific symbols.

It is useful because classical NLP models such as Bag of Words, TF-IDF, Naive Bayes, Logistic Regression, and SVM are sensitive to surface forms. For example, `Good`, `good`, and `GOOD` may be treated as three different features unless normalized.

Real-world uses include sentiment analysis, search engines, spam detection, chatbot logs, resume parsing, document classification, and information extraction.

2. Intuition

Think of preprocessing as cleaning ingredients before cooking. If the input text is messy, the model learns noise instead of useful patterns.

Example:

Raw:

```
Wow!!! This product is AMAZING :) Visit https://shop.com
```

Cleaned:

```
wow product amazing
```

The best preprocessing depends on the task. For sentiment analysis, removing `:)` may lose useful emotion. For search, preserving numbers and product codes may matter.

3. Prerequisites

- Python strings and regular expressions

- Basic NLP pipeline idea
- Tokenization
- Feature extraction
- Train/test split
- Understanding of data leakage

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Lowercasing	Convert text to lowercase	Reduces vocabulary size	Apple -> apple	When can lowercasing be harmful? Named entities.
Noise removal	Remove HTML, URLs, mentions, extra spaces	Prevents irrelevant tokens	 removed	Task-specific cleaning matters.
Punctuation handling	Remove or keep punctuation	Punctuation may carry meaning	great! vs great	Sentiment and dialogue may need punctuation.
Number handling	Keep, remove, or normalize numbers	Numbers can be critical	2026 -> <NUM>	Useful in finance, medical, resumes.
Unicode normalization	Standardize characters	Avoid duplicate forms	cafe vs cafe with accent	Common in multilingual text.
Spell correction	Fix typos	Helps matching	recieve -> receive	Can harm names and slang.

5. Algorithm / Working Process

1. Read raw text.
2. Normalize casing if useful.
3. Remove or replace URLs, emails, HTML tags, and mentions.
4. Normalize whitespace.
5. Tokenize text.
6. Optionally remove stopwords.
7. Optionally stem or lemmatize.
8. Convert text to numeric features.

Input: raw strings.

Processing: cleaning and normalization.

Output: cleaned strings or tokens.

Training process: preprocessing must be fitted only on training data if it learns anything, such as vocabulary.

Inference process: apply the same preprocessing steps used during training.

6. Mathematical Foundation

Text preprocessing has little direct math, but it affects vocabulary size:

$$V = \text{number of unique tokens}$$

Lowercasing, stopword removal, stemming, and lemmatization usually reduce V .

For sparse feature models:

$$X \in \mathbb{R}^{(N \times V)}$$

where:

- N = number of documents
- V = vocabulary size

Reducing noisy vocabulary reduces memory and overfitting risk.

7. Practical Implementation

```
import re
from html import unescape

def preprocess_text(text: str) -> str:
    text = unescape(text)
    text = text.lower()
    text = re.sub(r"https?://\S+|www\.\S+", " ", text)
    text = re.sub(r"<.*?>", " ", text)
    text = re.sub(r"^[^a-z0-9\s]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

texts = [
    "Wow!!! This PRODUCT is amazing :) Visit https://shop.com",
    "<b>Worst delivery</b> ever... arrived 5 days late!",
]

cleaned = [preprocess_text(t) for t in texts]
print(cleaned)
```

8. Code Explanation

`unescape` handles HTML entities. Lowercasing reduces duplicate tokens. Regex removes URLs, HTML tags, punctuation, and repeated whitespace. The function returns a clean string that can be passed to tokenizers or vectorizers.

9. Training / Evaluation

Preprocessing choices should be validated, not guessed. Compare model performance with different cleaning levels.

Useful metrics:

- Accuracy for balanced classification
- F1-score for imbalanced classification
- Precision/recall for spam, fraud, medical, or safety tasks

Avoid data leakage by fitting vocabulary, TF-IDF, scalers, and feature selectors only on the training set.

10. Complexity and Cost

For N documents with average length L , basic preprocessing is usually:

```
Time:  $O(NL)$   
Memory:  $O(NL)$  or  $O(V)$ 
```

It runs easily on CPU. GPU is not needed.

11. Common Use Cases

- Spam detection
- Sentiment analysis
- Search indexing
- Topic classification
- Resume parsing
- Customer support ticket routing
- Chatbot log cleaning

12. Common Mistakes

- Applying aggressive cleaning without checking task needs
- Removing negation words like `not`
- Lowercasing named entities when entity recognition matters
- Fitting vocabulary before train/test split
- Removing emojis in sentiment tasks
- Treating all languages like English

13. Edge Cases / Limitations

Preprocessing can destroy information. Examples:

- `US` vs `us`
- `C++` becoming `c`
- `not good` becoming `good` if stopwords are poorly removed
- Product codes, emails, legal references, and medical dosages being removed

14. Variations

Variation	What changes	When to use	Placement importance
Minimal cleaning	Only normalize spaces and obvious noise	Transformers, BERT, GPT	High
Aggressive cleaning	Remove punctuation, stopwords, numbers	Classical models	High
Domain-specific cleaning	Preserve domain tokens	Finance, medical, legal	Medium
Multilingual normalization	Unicode and language-aware rules	Multilingual NLP	Medium

15. Related Topics

- Tokenization: splits cleaned text into units.
- Stopword removal: removes frequent low-information tokens.
- Stemming and lemmatization: normalize word forms.
- Bag of Words and TF-IDF: convert text into sparse features.
- Transformers: often need less manual preprocessing because tokenizers are learned.

16. Interview Questions

1. What is text preprocessing? Answer: It is the process of cleaning and normalizing raw text before modeling.
2. Why is preprocessing important in NLP? Answer: It reduces noise, vocabulary size, and inconsistent representations.
3. Should we always lowercase text? Answer: No. Lowercasing can hurt tasks involving names, acronyms, or case-sensitive meaning.
4. Why can stopwords removal be dangerous? Answer: It may remove important words like `not`, changing meaning.
5. What is data leakage in preprocessing? Answer: Fitting preprocessing artifacts like vocabulary or TF-IDF on the full dataset before splitting.
6. Is preprocessing equally important for BERT? Answer: No. BERT uses its own tokenizer and expects relatively raw text with minimal cleaning.

7. How do you handle URLs? Answer: Remove them or replace them with a placeholder such as `<URL>` if URL presence is predictive.
8. Why normalize numbers? Answer: To reduce sparsity while preserving the fact that a number appeared.
9. What preprocessing is useful for sentiment analysis? Answer: Lowercasing, noise removal, preserving negation, and sometimes preserving emojis or punctuation.
10. How do you choose preprocessing steps? Answer: Through task understanding, validation experiments, and error analysis.

17. Practice Tasks

- Write a cleaner for tweets that preserves hashtags and emojis.
- Compare Logistic Regression accuracy with raw text vs cleaned text.
- Debug a model where `not bad` is classified as negative.
- Build a preprocessing pipeline using scikit-learn `Pipeline`.
- Experiment with keeping URLs as `<URL>` instead of removing them.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Tweet Cleaner and Classifier	Cleans social text and predicts sentiment	Python, scikit-learn	Sentiment140	Shows practical NLP pipeline skill
Resume Text Normalizer	Extracts and cleans resume text	Python, regex, spaCy	Resume dataset	Useful for placement/HR tech
Support Ticket Router	Cleans tickets and classifies categories	Pandas, TF-IDF, Logistic Regression	Customer support dataset	Strong applied ML project

19. Quick Revision

- Key idea: clean raw text before feature extraction or modeling.
- Main formula: vocabulary size `V` controls feature dimension.
- When to use: almost every NLP project, especially classical ML.
- Important metrics: downstream task metrics.
- Common traps: over-cleaning, leakage, removing negation.
- Interview one-liner: preprocessing should be task-aware, not automatic.

20. Final Cheat Sheet

Item	Summary
Definition	Cleaning and normalizing text
Input/output	Raw text -> clean text/tokens

Item	Summary
Main steps	lowercase, remove noise, normalize, tokenize
Key hyperparameters	Cleaning rules, token patterns
Metrics	Downstream accuracy, F1, precision, recall
Pros	Reduces noise and sparsity
Cons	Can remove useful meaning
Best use cases	Classical NLP and search pipelines

Tokenization

1. Overview

Tokenization splits text into smaller units called tokens. Tokens may be words, subwords, characters, or bytes. Tokenization is one of the most important steps in NLP because models do not process raw text directly; they process token IDs or token-derived features.

It is used in almost every NLP system: search, classification, translation, summarization, BERT, GPT, speech transcripts, and multimodal language models.

2. Intuition

Tokenization is like cutting a sentence into meaningful pieces.

```
"I love NLP" -> ["I", "love", "NLP"]
```

Subword tokenization handles unknown words better:

```
"unhappiness" -> ["un", "happiness"] or ["un", "happy", "ness"]
```

3. Prerequisites

- Python string processing
- Vocabulary concept
- Word frequency
- Unicode basics
- Neural network input embeddings

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Word tokenization	Split by words	Simple and interpretable	I am here -> 3 tokens	Fails on unknown words
Character tokenization	Split into characters	No OOV issue	cat -> c a t	Longer sequences
Subword tokenization	Split into frequent pieces	Handles rare words	playing -> play ##ing	Used by BERT/GPT
Vocabulary	Set of known tokens	Determines model input IDs	token -> integer	Fixed vocabulary issue
Special tokens	Control tokens	Mark padding, unknowns, start/end	[CLS], [PAD]	Important in transformers

5. Algorithm / Working Process

For word tokenization:

1. Normalize text.
2. Split using whitespace or rules.
3. Handle punctuation.
4. Return token list.

For subword tokenization:

1. Train a vocabulary from corpus.
2. Learn frequent character/subword merges.
3. Split new words into known subword units.
4. Map tokens to IDs.

Input: text.

Output: tokens and token IDs.

Training: tokenizer vocabulary may be learned from a corpus.

Inference: the same tokenizer converts text to IDs for the model.

6. Mathematical Foundation

Tokenization maps text to a sequence:

$$x = [t_1, t_2, \dots, t_n]$$

Then tokens are mapped to IDs:

$$\text{id}(t_i) \in \{0, 1, \dots, V-1\}$$

For embeddings:

$$E \in \mathbb{R}^{(V \times d)}$$
$$\text{embedding}(t_i) = E[\text{id}(t_i)]$$

where V is vocabulary size and d is embedding dimension.

7. Practical Implementation

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

text = "Tokenization helps models understand unseen words."
encoded = tokenizer(text, padding=True, truncation=True, return_tensors="pt")

print(tokenizer.tokenize(text))
print(encoded["input_ids"])
print(encoded["attention_mask"])
```

8. Code Explanation

`AutoTokenizer` loads the tokenizer used by BERT. `tokenize` shows subword tokens. Calling the tokenizer returns `input_ids` and `attention_mask`, which are the standard inputs for transformer models.

9. Training / Evaluation

Tokenizer quality affects sequence length, OOV handling, and downstream accuracy. Good evaluation includes:

- Average token length per sentence
- Unknown token rate
- Downstream validation performance
- Latency impact

For transformers, use the exact tokenizer paired with the pretrained model.

10. Complexity and Cost

Tokenization is usually CPU-based.

Time: $O(L)$

Memory: $O(V + L)$

where **L** is text length and **V** is vocabulary size.

Subword tokenization can increase sequence length, affecting transformer cost:

Attention cost: $O(n^2 d)$

11. Common Use Cases

- Text classification
- Machine translation
- Chatbots
- Search engines
- Large language models
- Named entity recognition
- Speech transcript processing

12. Common Mistakes

- Using a tokenizer different from the pretrained model
- Forgetting padding and truncation
- Treating subword tokens as full words
- Ignoring maximum sequence length
- Removing spaces incorrectly in languages where spacing matters differently

13. Edge Cases / Limitations

- Long documents may exceed model limits.
- Rare words can be split into many pieces.
- Tokenization can behave unexpectedly with emojis, code, URLs, and multilingual text.
- Word tokenizers struggle with languages without spaces.

14. Variations

Variation	What changes	When to use	Importance
Whitespace tokenizer	Split on spaces	Simple baselines	High for basics
Rule-based tokenizer	Uses language rules	Classical NLP	Medium
BPE	Learns merge rules	GPT-style models	High
WordPiece	Subword likelihood-based	BERT	High

Variation	What changes	When to use	Importance
SentencePiece	Language-independent subword tokenizer	Multilingual models	High
Byte-level BPE	Tokenizes bytes	Robust GPT tokenization	High

15. Related Topics

- Word embeddings: token IDs map to vectors.
- BERT and GPT: depend heavily on subword tokenization.
- Attention masks: distinguish real tokens from padding.
- Text preprocessing: should not break tokenizer assumptions.

16. Interview Questions

1. What is tokenization? Answer: Splitting text into units used by NLP models.
2. Why do transformers use subword tokenization? Answer: It balances vocabulary size and rare word handling.
3. What is OOV? Answer: Out-of-vocabulary, a token not present in the tokenizer vocabulary.
4. How does subword tokenization reduce OOV? Answer: Rare words are decomposed into known smaller units.
5. What is an attention mask? Answer: A binary mask indicating real tokens vs padding tokens.
6. Why is tokenizer-model mismatch bad? Answer: Token IDs will map to wrong embeddings and degrade performance.
7. What is padding? Answer: Adding special tokens so sequences in a batch have equal length.
8. What is truncation? Answer: Cutting sequences longer than the model maximum length.
9. Difference between BPE and WordPiece? Answer: BPE learns frequent merges; WordPiece chooses merges based on likelihood improvement.
10. Why can tokenization affect cost? Answer: More tokens mean longer sequences, and transformer attention scales quadratically.

17. Practice Tasks

- Compare whitespace tokenization and BERT tokenization.
- Count tokens for 100 text samples using GPT/BERT tokenizers.
- Test how tokenizer handles emojis, URLs, and code.
- Build a simple vocabulary from scratch.
- Visualize token length distribution.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Tokenizer Explorer	Shows token splits for different models	Python, Streamlit, Hugging Face	User text	Great demo for LLM understanding
OOV Analyzer	Measures unknown/fragmented tokens	Python, pandas	Domain corpus	Useful for domain NLP
Sequence Cost Estimator	Estimates LLM token cost	Python, tiktoken/HF	Any documents	Practical AI engineering value

19. Quick Revision

- Key idea: text -> tokens -> token IDs.
- Main formula: embedding lookup $E[id(t)]$.
- When to use: always before NLP modeling.
- Important metrics: OOV rate, average token length, downstream F1.
- Common traps: tokenizer mismatch, wrong padding, over-truncation.
- Interview one-liner: tokenization defines the model's actual view of text.

20. Final Cheat Sheet

Item	Summary
Definition	Splitting text into model-readable units
Input/output	Text -> tokens/IDs
Main steps	split, map to vocab, pad/truncate
Key hyperparameters	vocab size, max length
Metrics	OOV rate, length, task score
Pros	Enables modeling
Cons	Can fragment meaning
Best use cases	All NLP systems

Stemming

1. Overview

Stemming reduces words to crude base forms called stems. It removes prefixes or suffixes using rules, often without checking whether the resulting stem is a valid word.

Examples:

```

playing -> play
studies -> studi
connected -> connect

```

Stemming is useful in search engines, information retrieval, and classical NLP where exact word form is less important than matching related terms.

2. Intuition

Stemming says: "These words look related, so treat them similarly."

For search, a user searching **connect** may also want documents containing **connected**, **connecting**, and **connection**.

3. Prerequisites

- Tokenization
- Morphology basics
- Classical NLP features
- Bag of Words and TF-IDF

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Stem	Reduced word form	Groups related words	running -> run	Stem may not be valid word
Rule-based suffix stripping	Remove endings	Fast and simple	studies -> studi	Accuracy vs speed
Porter Stemmer	Popular English stemmer	Common baseline	relational -> relat	Know limitations
Snowball Stemmer	Improved stemmer	Supports more languages	English, German	Often better than Porter
Overstemming	Different meanings collapse	Causes false matches	universe , university	Precision loss
Understemming	Related words not collapsed	Causes missed matches	data , datum	Recall loss

5. Algorithm / Working Process

1. Tokenize text.
2. For each token, apply stemming rules.
3. Replace token with its stem.
4. Build features using stems.

Input: tokens.

Output: stems.

Training: no learning in common stemmers.

Inference: apply same stemmer.

6. Mathematical Foundation

Stemming is a deterministic mapping:

$$f(\text{word}) = \text{stem}$$

In a document-term matrix, multiple words map to one feature:

$$\text{count}(\text{stem}) = \sum \text{count}(\text{word}_i) \text{ for all } \text{word}_i \text{ where } f(\text{word}_i) = \text{stem}$$

This reduces feature dimensionality.

7. Practical Implementation

```
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()
tokens = ["playing", "played", "plays", "studies", "connection"]

stems = [stemmer.stem(token) for token in tokens]
print(list(zip(tokens, stems)))
```

8. Code Explanation

`PorterStemmer` applies rule-based suffix stripping. The list comprehension transforms each token into a stem. This is usually done after tokenization and before vectorization.

9. Training / Evaluation

Evaluate stemming through downstream performance. It may improve recall in search and classification but reduce precision when different words collapse incorrectly.

Metrics:

- Search: precision, recall, MAP, NDCG
- Classification: F1-score, accuracy

10. Complexity and Cost

Stemming is cheap.

```
Time: O(NL)
Memory: O(1) extra per token
```

No GPU is needed.

11. Common Use Cases

- Search engines
- Document retrieval
- Spam detection
- Topic classification
- Keyword matching

12. Common Mistakes

- Using stemming for transformer inputs
- Assuming stems are valid words
- Applying stemming to names and product codes
- Ignoring overstemming and understemming
- Using English stemmers on other languages

13. Edge Cases / Limitations

Stemming is rough. It may produce unnatural forms like `studi`, `relat`, or `comput`. It ignores context, grammar, and part of speech.

14. Variations

Variation	What changes	When to use	Importance
Porter Stemmer	Classic rule-based	English baselines	High
Snowball Stemmer	Improved and multilingual	IR systems	Medium
Lancaster Stemmer	More aggressive	Small vocabulary needs	Low
Lemmatization	Dictionary/context-based	Need valid words	High

15. Related Topics

- Lemmatization: more accurate normalization.
- TF-IDF: often combined with stemming.
- Search ranking: stemming improves recall.
- Tokenization: stemming works after tokenization.

16. Interview Questions

1. What is stemming? Answer: Reducing words to stems using rules.
2. Does stemming always produce valid words? Answer: No.
3. Stemming vs lemmatization? Answer: Stemming is rule-based and crude; lemmatization uses vocabulary and grammar.
4. Why use stemming? Answer: To reduce vocabulary and match related word forms.
5. What is overstemming? Answer: Incorrectly merging unrelated words.
6. What is understemming? Answer: Failing to merge related words.
7. Is stemming useful for BERT? Answer: Usually no; BERT expects natural text and uses subwords.
8. Where is stemming common? Answer: Search and classical NLP.
9. How do you evaluate stemming? Answer: Compare downstream metrics with and without it.
10. Can stemming hurt sentiment analysis? Answer: Yes, if it removes useful word nuance or breaks negation patterns.

17. Practice Tasks

- Stem a dataset and compare vocabulary size before and after.
- Build a TF-IDF classifier with and without stemming.
- Find examples of overstemming.
- Compare Porter and Snowball stemmers.
- Build a small search system using stemming.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Mini Search Engine	Retrieves documents with stemming	Python, TF-IDF	News articles	Shows IR basics
Stemmer Comparison Lab	Compares stemming algorithms	NLTK, pandas	Any corpus	Good interview demo
Classical Spam Filter	Uses stems with ML classifier	scikit-learn, NLTK	SMS Spam	Strong beginner NLP project

19. Quick Revision

- Key idea: reduce word variants to stems.
- Main formula: words mapping to same stem share counts.
- When to use: classical NLP and search.
- Important metrics: F1, precision, recall.
- Common traps: using with transformers, overstemming.
- Interview one-liner: stemming is fast word normalization but linguistically rough.

20. Final Cheat Sheet

Item	Summary
Definition	Rule-based word reduction
Input/output	Token -> stem
Main steps	tokenize, apply stemmer, vectorize
Key hyperparameters	Stemmer choice
Metrics	Downstream score
Pros	Fast, reduces sparsity
Cons	Crude, invalid stems
Best use cases	Search, BoW, TF-IDF

Lemmatization

1. Overview

Lemmatization reduces words to their dictionary base form, called a lemma. Unlike stemming, it considers vocabulary and often part of speech.

Examples:

```
better -> good  
running -> run  
was -> be  
cars -> car
```

It is useful when we want word normalization without producing invalid stems.

2. Intuition

Lemmatization asks: "What dictionary word does this form come from?"

Instead of chopping endings mechanically, it uses linguistic knowledge. **studies** becomes **study**, not **studi**.

3. Prerequisites

- Tokenization
- Parts of speech
- Morphology
- Classical ML text features
- Basic linguistic grammar

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Lemma	Dictionary base form	Valid normalized token	went -> go	More accurate than stemming
POS tagging	Identify grammatical role	Lemma can depend on POS	saw noun/verb	Context matters
Morphology	Word formation rules	Handles inflection	mice -> mouse	Language-specific
WordNet lemmatizer	Dictionary-based	Common in NLTK	cars -> car	Needs POS for best output
spaCy lemmatizer	Pipeline-based	Practical production use	token.lemma_	Better workflow

5. Algorithm / Working Process

1. Tokenize text.
2. Optionally perform POS tagging.
3. Look up or infer lemma for each token.
4. Replace tokens with lemmas.
5. Use lemmas for features or analysis.

Input: tokens.

Output: lemmas.

Training: usually no task-specific training.

Inference: apply the same lemmatizer pipeline.

6. Mathematical Foundation

Lemmatization is a mapping conditioned on context:

```
lemma = f(word, POS, context)
```

Vocabulary reduction:

```
V_lemma <= V_original
```

In sparse models, lower V means fewer feature dimensions.

7. Practical Implementation

```
import spacy

nlp = spacy.load("en_core_web_sm")

text = "The children were running faster than the adults."
doc = nlp(text)

for token in doc:
    print(token.text, token.pos_, token.lemma_)
```

8. Code Explanation

spaCy tokenizes, tags parts of speech, and lemmatizes in one pipeline. `token.text` is the original word, `token.pos_` is its part of speech, and `token.lemma_` is the base form.

9. Training / Evaluation

For classification, evaluate by downstream metrics. For linguistic tasks, compare lemmas to annotated gold labels.

Important checks:

- Does lemmatization reduce useful distinctions?
- Does it handle domain words?
- Does it work on the target language?

10. Complexity and Cost

Lemmatization is slower than stemming because it may require POS tagging.

```
Time: O(NL)
Memory: depends on model/dictionary
```

CPU is enough for most datasets.

11. Common Use Cases

- Search and retrieval
- Topic modeling
- Text classification
- Linguistic analysis
- Keyword extraction
- Document clustering

12. Common Mistakes

- Lemmatizing without POS when POS is needed

- Assuming lemmatization always improves performance
- Using it before BERT/GPT tokenization
- Applying English lemmatizers to multilingual text
- Removing domain meaning by over-normalizing

13. Edge Cases / Limitations

Lemmatizers may fail on slang, typos, names, code-mixed language, product names, and domain terms. POS tagging errors can cause wrong lemmas.

14. Variations

Variation	What changes	When to use	Importance
Dictionary-based	Uses lexicon	Simple English NLP	High
POS-aware	Uses part of speech	Better accuracy	High
Neural lemmatizer	Learns from data	Morphologically rich languages	Medium
Stemming	Rule-based alternative	Fast retrieval	High

15. Related Topics

- Stemming: faster but rougher.
- POS tagging: improves lemmatization.
- TF-IDF: lemmatized features reduce sparsity.
- Transformers: usually skip manual lemmatization.

16. Interview Questions

1. What is lemmatization? Answer: Converting words to dictionary base forms.
2. How is lemmatization different from stemming? Answer: Lemmatization is linguistically informed and returns valid words.
3. Why is POS important? Answer: The same word form can have different lemmas depending on grammar.
4. Example of lemmatization? Answer: `was` -> `be`, `mice` -> `mouse`.
5. Is lemmatization always better than stemming? Answer: Not always; it is slower and may not improve task metrics.
6. Should you lemmatize before BERT? Answer: Usually no.
7. What can go wrong? Answer: POS errors, domain terms, slang, multilingual issues.
8. How does lemmatization affect vocabulary? Answer: It reduces vocabulary size.
9. Where is it useful? Answer: Search, classical ML, topic modeling.
10. How do you evaluate it? Answer: Through downstream validation or gold lemma accuracy.

17. Practice Tasks

- Lemmatize a news corpus and compare vocabulary size.
- Compare stemming vs lemmatization in classification.
- Inspect POS-dependent lemma errors.
- Build a keyword extractor using lemmas.
- Try lemmatization on noisy social media text.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Lemma Search	Search documents using lemmas	spaCy, scikit-learn	Wikipedia subset	Strong IR basics
Topic Modeling Pipeline	Lemmatizes before LDA	spaCy, gensim	News corpus	Shows NLP preprocessing depth
Resume Skill Normalizer	Normalizes skill phrases	spaCy, regex	Resume data	HR tech relevance

19. Quick Revision

- Key idea: convert words to dictionary base forms.
- Main formula: $\text{lemma} = f(\text{word}, \text{POS}, \text{context})$.
- When to use: classical NLP needing valid normalized words.
- Important metrics: downstream F1, vocabulary size.
- Common traps: no POS, using before transformers.
- Interview one-liner: lemmatization is accurate word normalization using linguistic context.

20. Final Cheat Sheet

Item	Summary
Definition	Dictionary-form word normalization
Input/output	Word/token -> lemma
Main steps	tokenize, POS tag, lemmatize
Key hyperparameters	Lemmatizer/model choice
Metrics	Task score, lemma accuracy
Pros	Valid words, more accurate
Cons	Slower, language-dependent
Best use cases	Classical NLP, search, topic modeling

Stopword Removal

1. Overview

Stopword removal removes very common words such as **the**, **is**, **and**, and **of**. These words often contribute little to classical bag-based models.

It is useful in search, TF-IDF, topic modeling, and keyword extraction. It is less useful, and often harmful, for modern transformers.

2. Intuition

In many documents, words like **the** appear everywhere, so they do not help distinguish topics. Removing them can make important words stand out.

But in sentiment, **not** is critical:

```
not good
```

If **not** is removed, the meaning flips.

3. Prerequisites

- Tokenization
- Word frequency
- Text classification basics
- TF-IDF
- Understanding of negation

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Stopword	Frequent low-information word	Reduces noise	the , is	Task-specific
Custom stopwords	Domain-specific frequent terms	Better filtering	patient in hospital notes	Avoid removing labels
Negation words	Words reversing meaning	Must often be kept	not , never	Common trap
Frequency-based removal	Remove top-k frequent words	Corpus-specific	top 50 terms	Can remove useful domain words

5. Algorithm / Working Process

1. Tokenize text.
2. Load stopwords list.
3. Optionally edit list for task.

4. Remove tokens that appear in stopwords set.
5. Join or vectorize remaining tokens.

Input: tokens.

Output: filtered tokens.

6. Mathematical Foundation

If document vector uses term counts:

$$x_j = \text{count}(\text{term}_j, \text{document})$$

Stopword removal deletes selected dimensions:

$$V' = V - S$$

where S is the stopwords set.

TF-IDF already downweights common words:

$$\text{idf}(t) = \log(N / \text{df}(t))$$

So stopwords removal may be less necessary with TF-IDF.

7. Practical Implementation

```
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS

custom_keep = {"not", "no", "never"}
stopwords = ENGLISH_STOP_WORDS - custom_keep

def remove_stopwords(text: str) -> str:
    tokens = text.lower().split()
    tokens = [t for t in tokens if t not in stopwords]
    return " ".join(tokens)

print(remove_stopwords("This movie is not good and not enjoyable"))
```

8. Code Explanation

The code starts with scikit-learn's English stopword set, keeps negation words, then filters tokens. This simple version works for demos, but production systems should use proper tokenization.

9. Training / Evaluation

Compare with and without stopword removal. Track:

- Vocabulary size
- F1-score
- Recall loss
- Errors involving negation

Stopwords should be selected using training data only if generated from corpus frequencies.

10. Complexity and Cost

Set lookup is fast.

```
Time:  $O(\text{number of tokens})$   
Memory:  $O(\text{size of stopword set})$ 
```

11. Common Use Cases

- Keyword extraction
- Topic modeling
- Search indexing
- Classical text classification
- Word clouds

12. Common Mistakes

- Removing negation
- Removing words before understanding the task
- Using generic stopwords for domain-specific text
- Removing stopwords for BERT/GPT
- Creating stopwords from full dataset before splitting

13. Edge Cases / Limitations

Stopwords can be meaningful in:

- Sentiment
- Question answering
- Legal text
- Dialogue
- Natural language inference
- Machine translation

14. Variations

Variation	What changes	When to use	Importance
Fixed list	Use predefined stopwords	Quick baseline	High
Custom list	Add/remove words	Domain projects	High
Frequency-based	Remove most frequent terms	Large corpora	Medium
No removal	Keep all tokens	Transformers	High

15. Related Topics

- TF-IDF: automatically downweights common words.
- Bag of Words: stopwords removal reduces dimensions.
- Sentiment analysis: negation handling matters.
- Topic modeling: stopwords can dominate topics.

16. Interview Questions

1. What are stopwords? Answer: Common words often carrying little discriminative meaning.
2. Why remove stopwords? Answer: To reduce noise and vocabulary size in classical models.
3. When should stopwords not be removed? Answer: For transformers, sentiment, QA, translation, and tasks needing word order.
4. Why is removing **not** dangerous? Answer: It can reverse sentiment meaning.
5. Does TF-IDF need stopwords removal? Answer: Not always, because IDF downweights common words.
6. What are custom stopwords? Answer: Domain-specific frequent words that are not useful.
7. How do you evaluate stopwords removal? Answer: Compare validation metrics and error cases.
8. Can stopwords removal cause data leakage? Answer: Yes, if stopwords lists are learned from the full dataset.
9. Is stopwords removal language-specific? Answer: Yes.
10. How does it affect memory? Answer: It reduces vocabulary dimensions.

17. Practice Tasks

- Build a classifier with and without stopwords removal.
- Create a custom stopwords list for reviews.
- Analyze errors caused by removing negation.
- Compare TF-IDF with stopwords removed vs kept.
- Build a word cloud after stopwords filtering.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Topic Cleaner	Removes domain stopwords for topics	Python, sklearn	News articles	Demonstrates preprocessing judgement
Sentiment Negation Study	Shows effect of keeping/removing negation	sklearn	IMDb	Good interview talking point
Keyword Extractor	Extracts keywords after stopwords filtering	Python, TF-IDF	Research abstracts	Useful NLP mini-project

19. Quick Revision

- Key idea: remove frequent low-information words.
- Main formula: TF-IDF already downweights common terms.
- When to use: classical NLP and topic modeling.
- Important metrics: F1, vocabulary size.
- Common traps: removing negation, using with transformers.
- Interview one-liner: stopwords removal is task-dependent, not a default rule.

20. Final Cheat Sheet

Item	Summary
Definition	Removing common low-value words
Input/output	Tokens -> filtered tokens
Main steps	tokenize, load list, filter
Key hyperparameters	stopword list
Metrics	downstream score
Pros	Reduces noise and dimension
Cons	Can remove meaning
Best use cases	BoW, TF-IDF, topic models

Bag of Words

1. Overview

Bag of Words, or BoW, represents text by counting word occurrences. It ignores grammar and word order but captures which words appear and how often.

It is one of the simplest and most important NLP baselines. It is widely used in spam detection, sentiment classification, topic classification, and search.

2. Intuition

Imagine putting all words from a document into a bag and counting them. The order is forgotten.

```
"good movie good acting"
```

Vocabulary:

```
["acting", "good", "movie"]
```

Vector:

```
[1, 2, 1]
```

3. Prerequisites

- Tokenization
- Vocabulary
- Sparse matrices
- Basic classification
- Train/test split

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Vocabulary	Unique known words	Defines vector dimensions	10,000 words -> 10,000 features	Vocabulary must fit train only
Count vector	Word frequency vector	Model input	good:2	Sparse representation
N-grams	Consecutive token groups	Captures limited order	not good	Bigram fixes negation partly
Sparse matrix	Mostly zeros	Saves memory	CSR matrix	Important for scalability
Binary BoW	Presence instead of count	Useful for short texts	word exists or not	Count vs presence

5. Algorithm / Working Process

1. Preprocess text.
2. Tokenize documents.
3. Build vocabulary from training documents.

4. Count token occurrences per document.
5. Train ML model on count vectors.
6. For new text, use same vocabulary and transform to counts.

Input: documents.

Output: document-term matrix.

6. Mathematical Foundation

For N documents and vocabulary size V :

$$X \in \mathbb{R}^{(N \times V)}$$

Each element:

$$X[i, j] = \text{count}(\text{term}_j \text{ in } \text{document}_i)$$

For binary BoW:

$$X[i, j] = 1 \text{ if } \text{term}_j \text{ appears else } 0$$

A classifier such as Logistic Regression uses:

$$p(y=1|x) = \text{sigmoid}(w^T x + b)$$

7. Practical Implementation

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

texts = [
    "great movie with good acting",
    "bad movie and boring story",
    "excellent acting and great story",
    "boring film with bad acting",
]
labels = [1, 0, 1, 0]

model = Pipeline([
    ("bow", CountVectorizer(ngram_range=(1, 2))),
    ("clf", LogisticRegression(max_iter=1000)),
])
```

```
] )  
  
model.fit(texts, labels)  
print(model.predict(["great acting but boring story"]))
```

8. Code Explanation

`CountVectorizer` builds the vocabulary and creates count vectors. `ngram_range=(1, 2)` includes unigrams and bigrams. `Pipeline` ensures the same transformation is used during training and prediction.

9. Training / Evaluation

Use train/validation/test split. Common metrics:

- Accuracy for balanced data
- Precision/recall/F1 for imbalanced data
- Confusion matrix for error analysis

Important hyperparameters:

- `max_features`
- `min_df`
- `max_df`
- `ngram_range`
- `binary`

10. Complexity and Cost

```
Training vectorization: O(total tokens)  
Memory: O(nonzero counts)  
Classifier cost: depends on model and V
```

BoW is CPU-friendly and scalable with sparse matrices.

11. Common Use Cases

- Spam detection
- Sentiment analysis
- Intent classification
- Topic classification
- News categorization
- Search baselines

12. Common Mistakes

- Fitting vectorizer on test data
- Ignoring word order completely
- Creating huge vocabulary without pruning

- Not using sparse matrices
- Removing negation then expecting sentiment accuracy

13. Edge Cases / Limitations

BoW cannot capture:

- Long-range context
- Word order beyond n-grams
- Synonyms
- Polysemy
- Compositional meaning

good not and not good may look similar unless bigrams are used.

14. Variations

Variation	What changes	When to use	Importance
Binary BoW	Uses presence only	Short texts	Medium
Count BoW	Uses frequencies	General baseline	High
N-gram BoW	Adds phrases	Sentiment, intent	High
Character BoW	Uses char n-grams	Typos, languages	Medium
TF-IDF	Weighted BoW	Search/classification	High

15. Related Topics

- TF-IDF: weighted version of BoW.
- Naive Bayes: classic model for BoW.
- Logistic Regression: strong sparse text baseline.
- Word embeddings: dense semantic alternative.
- Transformers: contextual alternative.

16. Interview Questions

1. What is Bag of Words? Answer: A text representation based on word counts.
2. What does BoW ignore? Answer: Word order, grammar, and context.
3. Why is BoW sparse? Answer: Each document contains only a small subset of vocabulary.
4. What are n-grams? Answer: Contiguous groups of `n` tokens.
5. Why use bigrams? Answer: They capture short phrases like `not good`.
6. What is vocabulary leakage? Answer: Building vocabulary using test data.
7. BoW vs TF-IDF? Answer: BoW counts terms; TF-IDF weights terms by importance.

8. Can BoW handle synonyms? Answer: Poorly, because different words are separate features.
9. Which classifiers work well with BoW? Answer: Naive Bayes, Logistic Regression, Linear SVM.
10. Why is BoW still useful? Answer: It is simple, fast, interpretable, and strong for many classification tasks.

17. Practice Tasks

- Implement BoW manually for five sentences.
- Train spam classifier using `CountVectorizer`.
- Compare unigram vs bigram features.
- Tune `min_df` and `max_features`.
- Inspect top Logistic Regression coefficients.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
SMS Spam Classifier	Predicts spam using BoW	sklearn	SMS Spam Collection	Classic placement project
News Category Classifier	Classifies articles	sklearn	AG News	Shows sparse ML skill
Intent Classifier	Classifies chatbot intents	sklearn	CLINC150	Useful AI engineer baseline

19. Quick Revision

- Key idea: count words in a fixed vocabulary.
- Main formula: $X[i,j] = \text{count}(\text{term}_j, \text{doc}_i)$.
- When to use: fast classical NLP baseline.
- Important metrics: accuracy, F1.
- Common traps: leakage, huge vocabulary, no n-grams.
- Interview one-liner: BoW is simple, sparse, interpretable, and ignores order.

20. Final Cheat Sheet

Item	Summary
Definition	Count-based text representation
Input/output	Text -> sparse count vector
Main steps	tokenize, build vocab, count
Key hyperparameters	n-grams, max_features, min_df
Metrics	accuracy, F1
Pros	Fast, simple, interpretable

Item	Summary
Cons	Ignores context and semantics
Best use cases	Classification baselines

TF-IDF

1. Overview

TF-IDF stands for Term Frequency-Inverse Document Frequency. It represents documents by giving high weight to words that appear often in a document but not in many documents.

It improves over simple counts by reducing the importance of common words and highlighting discriminative words.

2. Intuition

A word is important if:

- It appears frequently in a document.
- It is rare across the corpus.

Example: In sports articles, **goal** may be more informative than **the**.

3. Prerequisites

- Bag of Words
- Term frequency
- Logarithms
- Sparse matrices
- Basic classification and retrieval

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
TF	Frequency in document	Captures local importance	goal appears 5 times	Raw vs normalized TF
DF	Number of documents containing term	Measures commonness	term in 900 docs	Needed for IDF
IDF	Inverse document frequency	Downweights common terms	low IDF for the	Formula often asked
TF-IDF score	TF times IDF	Final weight	high for rare frequent term	Better than counts

Subtopic	Meaning	Why it matters	Example	Interview angle
L2 normalization	Scale document vectors	Helps cosine similarity	unit norm vector	Retrieval relevance

5. Algorithm / Working Process

1. Build vocabulary from training corpus.
2. Count term frequency in each document.
3. Compute document frequency for each term.
4. Compute IDF.
5. Multiply TF by IDF.
6. Train classifier or compute similarities.

Input: documents.

Output: sparse weighted matrix.

6. Mathematical Foundation

Term frequency:

$$tf(t, d) = \text{count}(t, d)$$

Document frequency:

$$df(t) = \text{number of documents containing } t$$

Inverse document frequency:

$$idf(t) = \log((N + 1) / (df(t) + 1)) + 1$$

TF-IDF:

$$tfidf(t, d) = tf(t, d) * idf(t)$$

Cosine similarity:

$$\cos(x, y) = (x \cdot y) / (||x|| ||y||)$$

7. Practical Implementation

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.pipeline import Pipeline

texts = [
    "free money now",
    "limited offer free prize",
    "meeting schedule confirmed",
    "project meeting tomorrow",
]
labels = [1, 1, 0, 0]

clf = Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1, 2), min_df=1)),
    ("model", LogisticRegression(max_iter=1000)),
])

clf.fit(texts, labels)
print(clf.predict(["free prize offer"]))
```

8. Code Explanation

`TfidfVectorizer` handles tokenization, vocabulary creation, TF calculation, IDF calculation, and normalization. `LogisticRegression` learns a linear decision boundary over TF-IDF features.

9. Training / Evaluation

Fit TF-IDF only on training data. Evaluate using validation/test sets.

Metrics:

- Classification: accuracy, F1, ROC-AUC
- Retrieval: precision@k, recall@k, MAP, NDCG

Important hyperparameters:

- `ngram_range`
- `max_features`
- `min_df`
- `max_df`
- `sublinear_tf`
- `norm`

10. Complexity and Cost

Vectorization time: $O(\text{total tokens})$

```
Memory: 0(nonzero entries)
```

TF-IDF is fast on CPU and suitable for large baselines.

11. Common Use Cases

- Search ranking
- Document similarity
- Spam detection
- Resume-job matching
- FAQ retrieval
- News classification

12. Common Mistakes

- Fitting TF-IDF before train/test split
- Assuming high TF-IDF means semantic importance
- Ignoring n-grams
- Not tuning `min_df` and `max_df`
- Using TF-IDF for tasks needing deep context

13. Edge Cases / Limitations

TF-IDF cannot understand synonyms. `car` and `automobile` are unrelated unless both occur. It also ignores word order except through n-grams and cannot handle deep semantics.

14. Variations

Variation	What changes	When to use	Importance
Sublinear TF	Uses $1 + \log(\text{tf})$	Long documents	Medium
Character TF-IDF	Character n-grams	Typos, names	Medium
BM25	Improved retrieval weighting	Search engines	High
TF-IDF + SVM	Strong classifier	Text classification	High

15. Related Topics

- BoW: TF-IDF is weighted BoW.
- BM25: stronger search weighting.
- Cosine similarity: common for TF-IDF retrieval.
- Word embeddings: dense semantic alternative.
- RAG: TF-IDF/BM25 can be lexical retrievers.

16. Interview Questions

1. What is TF-IDF? Answer: A weighting method combining term frequency and inverse document frequency.

2. Why use IDF? Answer: To reduce weight of common corpus-wide terms.
3. Write the TF-IDF formula. Answer: $\text{tfidf}(t,d)=\text{tf}(t,d)*\log(N/\text{df}(t))$, often smoothed.
4. How is TF-IDF different from BoW? Answer: BoW counts terms; TF-IDF weights them.
5. What is cosine similarity? Answer: Dot product normalized by vector magnitudes.
6. Why is TF-IDF sparse? Answer: Each document contains few vocabulary terms.
7. What is `max_df`? Answer: It removes terms appearing in too many documents.
8. What is `min_df`? Answer: It removes rare terms below a document-frequency threshold.
9. Can TF-IDF capture semantics? Answer: Not well; it is lexical.
10. Where is TF-IDF still useful? Answer: Baselines, search, classification, and retrieval.

17. Practice Tasks

- Implement TF-IDF from scratch using NumPy.
- Build document similarity search.
- Compare CountVectorizer vs TfidfVectorizer.
- Tune `min_df`, `max_df`, and `ngram_range`.
- Inspect top TF-IDF terms per document.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
FAQ Search Engine	Retrieves similar FAQs	sklearn	FAQ dataset	Direct RAG baseline
Resume Matcher	Matches resumes to jobs	TF-IDF, cosine	Resume/job descriptions	Placement-relevant
News Similarity Tool	Finds related articles	sklearn, Streamlit	AG News	Shows search and NLP

19. Quick Revision

- Key idea: frequent in document, rare in corpus means important.
- Main formula: $\text{tfidf} = \text{tf} * \text{idf}$.
- When to use: lexical retrieval and classical classification.
- Important metrics: F1, precision@k, NDCG.
- Common traps: data leakage, confusing lexical with semantic similarity.
- Interview one-liner: TF-IDF is a strong sparse baseline for lexical importance.

20. Final Cheat Sheet

Item	Summary
Definition	Weighted word-frequency representation
Input/output	Text -> sparse weighted vector
Main steps	count TF, compute IDF, multiply
Key hyperparameters	n-grams, min_df, max_df, norm
Metrics	F1, precision@k, NDCG
Pros	Fast, interpretable, strong baseline
Cons	No deep semantics
Best use cases	Search and text classification

Word Embeddings

1. Overview

Word embeddings represent words as dense vectors in a continuous space. Unlike BoW or TF-IDF, embeddings can capture semantic similarity.

Example:

```
vector("king") close to vector("queen")  
vector("cat") close to vector("dog")
```

They are used in text classification, search, recommendation, translation, chatbots, and neural NLP models.

2. Intuition

Words that appear in similar contexts tend to have similar meanings. This is called the distributional hypothesis:

```
"You shall know a word by the company it keeps."
```

If **doctor** and **nurse** often occur near words like **hospital**, **patient**, and **medicine**, their vectors become close.

3. Prerequisites

- Linear algebra
- Vectors and dot products
- Neural networks
- Tokenization

- Gradient descent
- Probability basics

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Dense vector	Low-dimensional real vector	Captures similarity	300-d vector	Dense vs sparse
Embedding matrix	Lookup table for tokens	Neural input layer	$E \in \mathbb{R}^{(V \times d)}$	Learnable parameters
Semantic similarity	Nearby vectors have related meaning	Useful for search	<i>car, vehicle</i>	Cosine similarity
Static embedding	One vector per word	Simple and reusable	Word2Vec, GloVe	Cannot handle context
Contextual embedding	Vector depends on sentence	Handles polysemy	BERT	Modern NLP

5. Algorithm / Working Process

For static embeddings:

1. Build vocabulary.
2. Initialize embedding matrix.
3. Train using co-occurrence or prediction objective.
4. Use learned vectors in downstream models.

Input: token IDs.

Processing: embedding lookup.

Output: dense vectors.

6. Mathematical Foundation

Embedding matrix:

$$E \in \mathbb{R}^{(V \times d)}$$

For token ID i :

$$e_i = E[i]$$

Cosine similarity:

$$\cos(e_i, e_j) = (e_i \cdot e_j) / (||e_i|| ||e_j||)$$

Neural models update embeddings using backpropagation:

$$E \leftarrow E - \eta * dL/dE$$

7. Practical Implementation

```
import torch
from torch import nn

vocab_size = 1000
embedding_dim = 50

embedding = nn.Embedding(vocab_size, embedding_dim)

token_ids = torch.tensor([[4, 10, 25], [8, 9, 2]])
vectors = embedding(token_ids)

print(vectors.shape) # batch_size, sequence_length, embedding_dim
```

8. Code Explanation

`nn.Embedding` stores a learnable matrix of shape `vocab_size x embedding_dim`. Passing token IDs returns dense vectors for each token. These vectors can be fed into RNNs, LSTMs, CNNs, or Transformers.

9. Training / Evaluation

Embeddings can be:

- Trained from scratch
- Loaded from pretrained Word2Vec/GloVe
- Fine-tuned during task training
- Frozen to reduce overfitting

Evaluation:

- Downstream task accuracy/F1
- Word similarity benchmarks
- Analogy tasks
- Retrieval quality

10. Complexity and Cost

Embedding lookup:

```
Time:  $O(\text{sequence\_length})$   
Memory:  $O(Vd)$ 
```

Large vocabularies can consume significant memory.

11. Common Use Cases

- Text classification
- Named entity recognition
- Machine translation
- Semantic search
- Recommendation
- Clustering words/documents

12. Common Mistakes

- Confusing sparse vectors with embeddings
- Using random embeddings with too little data
- Ignoring OOV tokens
- Averaging embeddings when order matters
- Assuming static embeddings solve polysemy

13. Edge Cases / Limitations

Static embeddings give one vector per word, so **bank** in **river bank** and **bank loan** has the same vector. They also struggle with rare words, domain-specific terms, and evolving language.

14. Variations

Variation	What changes	When to use	Importance
Word2Vec	Predictive local context	General semantic embeddings	High
GloVe	Global co-occurrence factorization	Pretrained embeddings	High
FastText	Uses character n-grams	Rare/morphological words	Medium
Contextual embeddings	Context-dependent vectors	Modern NLP	High

15. Related Topics

- Word2Vec and GloVe: popular static embeddings.
- RNN/LSTM: consume embeddings as sequence inputs.
- Transformers: learn contextual embeddings.
- Semantic search: embeddings enable vector similarity.

16. Interview Questions

1. What are word embeddings? Answer: Dense vector representations of words.
2. Why are embeddings better than one-hot vectors? Answer: They are dense and capture similarity.
3. What is an embedding matrix? Answer: A learnable lookup table mapping token IDs to vectors.
4. What is cosine similarity used for? Answer: Measuring vector similarity independent of magnitude.
5. Static vs contextual embeddings? Answer: Static has one vector per word; contextual depends on sentence.
6. What is OOV? Answer: A word not in vocabulary.
7. Why can embeddings capture analogies? Answer: Vector directions can encode relationships.
8. Can embeddings be trained end-to-end? Answer: Yes, as model parameters.
9. When freeze embeddings? Answer: Small datasets or to reduce overfitting.
10. Limitation of static embeddings? Answer: They cannot handle polysemy well.

17. Practice Tasks

- Train a small embedding layer in PyTorch.
- Average word embeddings for sentence classification.
- Compute cosine similarities between words.
- Compare random vs pretrained embeddings.
- Visualize embeddings using t-SNE or PCA.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Semantic Word Explorer	Finds similar words	Gensim, Streamlit	Word2Vec/GloVe	Shows embedding intuition
Review Classifier	Uses averaged embeddings	PyTorch	IMDb	Neural NLP basics
Job Skill Similarity	Embeds skill terms	Python, cosine similarity	Job posts	Placement-friendly application

19. Quick Revision

- Key idea: words become dense semantic vectors.
- Main formula: $\mathbf{e}_i = \mathbf{E}[\mathbf{i}]$.
- When to use: neural NLP and semantic similarity.
- Important metrics: downstream F1, cosine similarity.
- Common traps: OOV, static polysemy.
- Interview one-liner: embeddings turn discrete tokens into trainable semantic vectors.

20. Final Cheat Sheet

Item	Summary
Definition	Dense vector representation of tokens
Input/output	Token ID -> vector
Main steps	build vocab, lookup vector, train/freeze
Key hyperparameters	embedding dimension, vocab size
Metrics	downstream score, similarity
Pros	Captures semantics
Cons	Static versions ignore context
Best use cases	Neural NLP and semantic search

Word2Vec

1. Overview

Word2Vec is a neural method for learning word embeddings from text. It learns vectors by predicting words from nearby context or predicting context from a word.

Two main architectures:

- CBOW: predicts target word from surrounding context.
- Skip-gram: predicts surrounding context words from target word.

2. Intuition

If two words appear in similar neighborhoods, they should have similar vectors.

Example:

```
The doctor treated the patient.  
The nurse treated the patient.
```

doctor and **nurse** appear in similar contexts, so Word2Vec places them close.

3. Prerequisites

- Word embeddings
- Neural networks
- Softmax
- Cross-entropy loss
- Gradient descent

- Negative sampling

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
CBOW	Context -> target	Faster, good for frequent words	predict cat from neighbors	Compare with Skip-gram
Skip-gram	Target -> context	Better for rare words	cat predicts sat , mat	Common interview favorite
Window size	Context range	Controls local meaning	2 words left/right	Small vs large window
Negative sampling	Train against fake words	Speeds training	positive pair + negatives	Formula often asked
Hierarchical softmax	Tree-based softmax	Efficient large vocab	Huffman tree	Less common now

5. Algorithm / Working Process

Skip-gram process:

1. Tokenize corpus.
2. Build vocabulary.
3. For each center word, collect context words within window.
4. Create positive pairs (**center**, **context**).
5. Sample negative words not in context.
6. Train embeddings to score positive pairs high and negative pairs low.

Input: center word.

Output: probability or score for context words.

Training: optimize prediction objective.

Inference: use learned word vectors.

6. Mathematical Foundation

Skip-gram objective:

```
maximize sum log P(context | center)
```

Softmax probability:

$$P(w_o \mid w_i) = \exp(v'_{wo} \cdot v_{wi}) / \sum_w \exp(v'_w \cdot v_{wi})$$

Negative sampling loss for one positive pair:

$$L = -\log \sigma(v'_o \cdot v_i) - \sum_{k=1}^K \log \sigma(-v'_k \cdot v_i)$$

where:

- v_i = input vector for center word
- v'_o = output vector for true context word
- v'_k = output vector for negative sample
- K = number of negative samples

7. Practical Implementation

```
from gensim.models import Word2Vec

sentences = [
    ["doctor", "treated", "patient"],
    ["nurse", "treated", "patient"],
    ["teacher", "taught", "student"],
    ["student", "read", "book"],
]

model = Word2Vec(
    sentences=sentences,
    vector_size=50,
    window=2,
    min_count=1,
    sg=1,
    workers=1,
    epochs=100,
)

print(model.wv.most_similar("doctor", topn=3))
```

8. Code Explanation

`vector_size` controls embedding dimension. `window` controls context size. `sg=1` selects Skip-gram; `sg=0` selects CBOW. `min_count=1` keeps all words in this tiny example.

9. Training / Evaluation

Important hyperparameters:

- `vector_size`

- window
- min_count
- negative
- epochs
- sg

Evaluation:

- Word similarity
- Analogy tasks
- Downstream model performance

10. Complexity and Cost

With negative sampling:

```
Time per pair:  $O(Kd)$   
Memory:  $O(Vd)$ 
```

where K is negative samples and d is embedding dimension.

Word2Vec trains efficiently on CPU for moderate corpora.

11. Common Use Cases

- Semantic similarity
- Recommendation
- Query expansion
- Feature initialization
- Clustering terms
- Domain-specific embeddings

12. Common Mistakes

- Training on too small a corpus
- Using tiny window without reason
- Ignoring rare word handling
- Confusing CBOW and Skip-gram
- Expecting contextual meaning

13. Edge Cases / Limitations

Word2Vec gives one vector per word, so it cannot distinguish different meanings of the same word. It also needs enough corpus data and struggles with OOV words.

14. Variations

Variation	What changes	When to use	Importance
CBOW	Predict word from context	Faster training	High
Skip-gram	Predict context from word	Rare words, quality	High
Negative sampling	Binary objectives	Large vocab	High
FastText	Adds subword n-grams	Rare words	Medium

15. Related Topics

- GloVe: global co-occurrence alternative.
- Embeddings: Word2Vec is a method to learn them.
- Neural language models: Word2Vec is an early predictive representation learner.
- Transformer embeddings: contextual successor.

16. Interview Questions

1. What is Word2Vec? Answer: A neural method to learn word embeddings from context.
2. CBOW vs Skip-gram? Answer: CBOW predicts target from context; Skip-gram predicts context from target.
3. Which handles rare words better? Answer: Skip-gram usually.
4. Why use negative sampling? Answer: To avoid expensive full softmax over vocabulary.
5. What is window size? Answer: Number of neighboring words used as context.
6. What is the training objective? Answer: Maximize probability of real word-context pairs.
7. What is a limitation? Answer: One vector per word, no context-specific meaning.
8. Does Word2Vec use labels? Answer: No, it is self-supervised.
9. What does cosine similarity measure? Answer: Directional similarity between embeddings.
10. Word2Vec vs TF-IDF? Answer: Word2Vec is dense semantic; TF-IDF is sparse lexical.

17. Practice Tasks

- Train Word2Vec on a small corpus.
- Compare CBOW and Skip-gram.
- Change window size and inspect neighbors.
- Use Word2Vec vectors in a classifier.
- Visualize vectors with PCA.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Domain Embedding Trainer	Trains Word2Vec on domain text	Gensim	Medical/legal corpus	Shows representation learning
Similar Skill Finder	Finds related resume skills	Gensim, cosine	Job descriptions	Placement-relevant
News Word Map	Visualizes related terms	Word2Vec, PCA	News corpus	Good explainable demo

19. Quick Revision

- Key idea: learn embeddings by predicting context.
- Main formula: negative sampling loss.
- When to use: static semantic word vectors.
- Important metrics: similarity, downstream F1.
- Common traps: small data, expecting context awareness.
- Interview one-liner: Word2Vec learns meaning from neighboring words.

20. Final Cheat Sheet

Item	Summary
Definition	Predictive word embedding method
Input/output	Corpus -> word vectors
Main steps	create context pairs, train, use vectors
Key hyperparameters	window, vector_size, negative, min_count
Metrics	similarity, analogy, task score
Pros	Fast, semantic
Cons	Static, OOV issues
Best use cases	Similarity and feature initialization

GloVe

1. Overview

GloVe, or Global Vectors, is a word embedding method based on global word co-occurrence statistics. It learns vectors so that their dot products approximate logarithms of co-occurrence counts.

It combines the global matrix-factorization idea with the usefulness of dense embeddings.

2. Intuition

If two words often occur together or share similar co-occurrence patterns, their vectors should reflect that relationship.

For example, **ice** co-occurs with **cold**, while **steam** co-occurs with **hot**. Ratios of co-occurrence probabilities encode meaning.

3. Prerequisites

- Word embeddings
- Co-occurrence matrix
- Dot product
- Loss minimization
- Gradient descent

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Co-occurrence matrix	Counts word-context pairs	Captures global corpus stats	X_{ij}	GloVe key idea
Weighted least squares	Training objective	Reduces effect of huge counts	weighting function	Know formula
Global context	Uses entire corpus statistics	Different from local prediction	matrix counts	GloVe vs Word2Vec
Static vectors	One vector per word	Simple reuse	pretrained GloVe	Limitation

5. Algorithm / Working Process

1. Build vocabulary.
2. Construct word-word co-occurrence matrix.
3. Initialize word and context vectors.
4. Optimize objective matching dot products to log co-occurrence counts.
5. Use trained word vectors downstream.

Input: corpus.

Output: dense word vectors.

6. Mathematical Foundation

GloVe objective:

$$J = \sum_{i,j=1}^V f(X_{ij}) (w_i^T w_{j_tilde} + b_i + b_{j_tilde} - \log X_{ij})^2$$

where:

- X_{ij} = co-occurrence count between word i and context j
- w_i, w_{j_tilde} = word and context vectors
- b_i, b_{j_tilde} = biases
- $f(X_{ij})$ = weighting function

Common weighting:

$$f(x) = (x / x_{\max})^{\alpha} \text{ if } x < x_{\max} \text{ else } 1$$

7. Practical Implementation

```
import numpy as np

# Tiny example: use pretrained vectors in real projects.
vectors = {
    "king": np.array([0.8, 0.6]),
    "queen": np.array([0.75, 0.65]),
    "car": np.array([-0.7, 0.2]),
}

def cosine(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

print(cosine(vectors["king"], vectors["queen"]))
print(cosine(vectors["king"], vectors["car"]))
```

8. Code Explanation

The example demonstrates how GloVe vectors are typically used after training: load vectors, compute cosine similarity, and use them as semantic features.

9. Training / Evaluation

GloVe training requires a large corpus and co-occurrence matrix. Most projects use pretrained vectors.

Evaluation:

- Word similarity
- Analogy tasks
- Downstream classification/retrieval metrics

10. Complexity and Cost

Memory depends on nonzero co-occurrence entries:

Memory: $O(\text{nnz}(X) + Vd)$
Training: $O(\text{nnz}(X) * d)$

Large corpora require efficient sparse storage.

11. Common Use Cases

- Pretrained word features
- Semantic similarity
- Text classification
- Search expansion
- Clustering words
- Initialization for neural models

12. Common Mistakes

- Confusing GloVe with Word2Vec
- Training GloVe on tiny datasets
- Ignoring OOV words
- Expecting context-specific embeddings
- Using mismatched embedding dimensions

13. Edge Cases / Limitations

GloVe is static and cannot distinguish word meaning by sentence context. It also depends on corpus quality and may encode social biases from training text.

14. Variations

Variation	What changes	When to use	Importance
Pretrained GloVe	Use public vectors	Fast projects	High
Domain GloVe	Train on domain corpus	Domain-specific vocabulary	Medium
FastText	Adds subword information	Rare words	Medium
Contextual embeddings	BERT/GPT-style	Modern NLP	High

15. Related Topics

- Word2Vec: predictive local context method.
- Matrix factorization: GloVe resembles weighted factorization.
- Embeddings: GloVe produces static embeddings.
- BERT: contextual alternative.

16. Interview Questions

1. What is GloVe? Answer: A word embedding method based on global co-occurrence statistics.

2. GloVe vs Word2Vec? Answer: GloVe uses global co-occurrence matrix; Word2Vec uses local prediction.
3. What does x_{ij} mean? Answer: Co-occurrence count of word i with context word j .
4. Why use log co-occurrence? Answer: To compress large count ranges.
5. What is the GloVe objective? Answer: Minimize weighted squared error between vector dot products and log co-occurrence.
6. Is GloVe supervised? Answer: No, it is unsupervised/self-supervised from text statistics.
7. What is a limitation? Answer: Static word meanings.
8. Why use weighting function? Answer: To prevent very frequent pairs from dominating.
9. Can GloVe handle OOV words? Answer: Not directly.
10. Where is GloVe useful? Answer: Pretrained semantic features and classical neural NLP baselines.

17. Practice Tasks

- Load pretrained GloVe vectors.
- Compute nearest neighbors using cosine similarity.
- Average GloVe vectors for document classification.
- Compare GloVe and TF-IDF on sentiment classification.
- Analyze biased nearest neighbors.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
GloVe Similarity Search	Finds similar words/docs	NumPy, GloVe	Pretrained vectors	Shows semantic vector skill
Review Classifier	Uses averaged GloVe vectors	sklearn/PyTorch	IMDb	Good NLP baseline
Bias Explorer	Studies embedding bias	Python, PCA	GloVe	Research internship relevance

19. Quick Revision

- Key idea: learn vectors from global co-occurrence.
- Main formula: weighted squared error on $w_i^T w_j$.
- When to use: pretrained static word vectors.
- Important metrics: similarity, downstream score.
- Common traps: confusing with Word2Vec, OOV.
- Interview one-liner: GloVe factorizes global word co-occurrence into semantic vectors.

20. Final Cheat Sheet

Item	Summary
Definition	Global co-occurrence word embedding method
Input/output	Corpus matrix -> word vectors
Main steps	build co-occurrence, optimize, use vectors
Key hyperparameters	vector size, window, x_max, alpha
Metrics	similarity, analogy, task score
Pros	Captures global statistics
Cons	Static, OOV, memory-heavy
Best use cases	Pretrained embeddings

RNN for Text

1. Overview

Recurrent Neural Networks, or RNNs, process sequences one token at a time while maintaining a hidden state. For text, RNNs read token embeddings sequentially and update memory at each step.

They are used historically for language modeling, sentiment analysis, machine translation, sequence labeling, and text generation. Transformers have mostly replaced them for large-scale NLP, but RNNs remain important for interviews and sequence modeling basics.

2. Intuition

An RNN reads a sentence like a person reading left to right. At each word, it remembers information from previous words.

"The movie was not good"

When it reaches **good**, the hidden state should remember **not**.

3. Prerequisites

- Neural networks
- Word embeddings
- Matrix multiplication
- Backpropagation
- Cross-entropy loss
- Sequence data

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Hidden state	Memory vector	Carries past information	h_t	Core RNN mechanism
Recurrent weight	Shared across time	Handles variable lengths	w_{hh}	Parameter sharing
Many-to-one	Sequence $\rightarrow$ label	Sentiment classification	final hidden state	Common task
Many-to-many	Sequence $\rightarrow$ sequence	POS tagging	output each step	NER/translation
Vanishing gradient	Gradients shrink over time	Hard long dependencies	forget early words	Why LSTM exists

5. Algorithm / Working Process

For text classification:

1. Tokenize sentence.
2. Convert tokens to IDs.
3. Look up embeddings.
4. Feed embeddings into RNN sequentially.
5. Use final hidden state.
6. Pass through classifier.
7. Train with cross-entropy.

Input: sequence of token IDs.

Output: class probability or token-level outputs.

6. Mathematical Foundation

Basic RNN update:

$$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b_h)$$

Output:

$$y_t = W_{hy} h_t + b_y$$

For classification:

$$p = \text{softmax}(W h_T + b)$$

Cross-entropy loss:

$$L = -\sum_i y_i \log(p_i)$$

Vanishing gradient occurs because repeated multiplication by recurrent weights can shrink gradients.

7. Practical Implementation

```
import torch
from torch import nn

class TextRNN(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, num_classes):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.rnn = nn.RNN(embed_dim, hidden_dim, batch_first=True)
        self.classifier = nn.Linear(hidden_dim, num_classes)

    def forward(self, input_ids):
        x = self.embedding(input_ids)
        output, hidden = self.rnn(x)
        final_hidden = hidden[-1]
        return self.classifier(final_hidden)

model = TextRNN(vocab_size=5000, embed_dim=100, hidden_dim=128, num_classes=2)
batch = torch.randint(1, 5000, (4, 20))
logits = model(batch)
print(logits.shape)
```

8. Code Explanation

The embedding layer converts token IDs into dense vectors. `nn.RNN` processes the sequence. `hidden[-1]` gives the last hidden state, which is passed to a linear classifier.

9. Training / Evaluation

Dataset preparation:

- Tokenize text.
- Build vocabulary.
- Pad/truncate sequences.
- Split train/validation/test.

Metrics:

- Accuracy

- F1-score
- Precision/recall

Improvements:

- Use LSTM/GRU
- Use bidirectional RNN
- Add dropout
- Use pretrained embeddings

10. Complexity and Cost

For sequence length T , hidden size H , input size D :

```
Time:  $O(T(H^2 + HD))$   
Memory:  $O(TH)$ 
```

RNNs are sequential and harder to parallelize than transformers.

11. Common Use Cases

- Sentiment classification
- Language modeling
- Named entity recognition
- POS tagging
- Text generation
- Time-series text-like sequences

12. Common Mistakes

- Ignoring padding masks
- Using final hidden state for long documents without attention
- Not handling vanishing gradients
- Forgetting `batch_first=True`
- Comparing RNNs unfairly with modern transformers

13. Edge Cases / Limitations

RNNs struggle with long-range dependencies, are slow to train on long sequences, and cannot parallelize over time steps easily.

14. Variations

Variation	What changes	When to use	Importance
Vanilla RNN	Simple recurrence	Learning basics	High
LSTM	Gates and cell state	Long dependencies	High

Variation	What changes	When to use	Importance
GRU	Simpler gated RNN	Efficient sequence modeling	Medium
Bidirectional RNN	Reads both directions	Classification/NER	High
Attention RNN	Adds weighted context	Seq2seq	Medium

15. Related Topics

- LSTM: solves vanishing gradient better.
- Attention: helps focus on relevant tokens.
- Transformer: replaces recurrence with self-attention.
- Word embeddings: RNN input representation.

16. Interview Questions

1. What is an RNN? Answer: A neural network for sequential data using recurrent hidden states.
2. Why are RNNs suitable for text? Answer: Text is a sequence and RNNs process tokens in order.
3. Write the RNN update equation. Answer: $h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b)$.
4. What is hidden state? Answer: A memory vector summarizing previous tokens.
5. What is vanishing gradient? Answer: Gradients become very small through many time steps.
6. Why did LSTM improve RNNs? Answer: It uses gates and cell state to preserve information.
7. Many-to-one example? Answer: Sentiment classification.
8. Many-to-many example? Answer: NER or POS tagging.
9. Why are RNNs slow? Answer: They process tokens sequentially.
10. RNN vs Transformer? Answer: RNN uses recurrence; Transformer uses parallel self-attention.

17. Practice Tasks

- Implement a sentiment classifier using `nn.RNN`.
- Compare RNN and LSTM on the same dataset.
- Plot validation loss for different sequence lengths.
- Add bidirectionality.
- Debug padding-related performance issues.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
RNN Sentiment Classifier	Classifies reviews	PyTorch	IMDb	Shows sequence basics

Project	What it does	Tech stack	Dataset	Resume value
Character Name Generator	Generates names character by character	PyTorch	Name dataset	Good generative basics
POS Tagger	Tags each word	PyTorch	CoNLL	Sequence labeling skill

19. Quick Revision

- Key idea: process sequence using hidden state.
- Main formula: $h_t = \tanh(Wx_t + Uh_{t-1} + b)$.
- When to use: sequence basics, small tasks.
- Important metrics: F1, accuracy, perplexity.
- Common traps: vanishing gradients, padding.
- Interview one-liner: RNNs read text step by step while carrying memory.

20. Final Cheat Sheet

Item	Summary
Definition	Recurrent sequence neural network
Input/output	Token sequence -> labels or sequence
Main steps	embed, recurrent update, classify
Key hyperparameters	hidden size, layers, dropout
Metrics	accuracy, F1, perplexity
Pros	Handles variable-length sequences
Cons	Slow, weak long memory
Best use cases	Sequence learning basics

LSTM for Text

1. Overview

Long Short-Term Memory networks, or LSTMs, are gated RNNs designed to handle long-range dependencies better than vanilla RNNs. They use a cell state and gates to decide what to remember, forget, and output.

They are used for sentiment analysis, NER, speech, translation, and sequence classification, especially before transformers became dominant.

2. Intuition

An LSTM is like a reader with a notebook. It can erase irrelevant information, write important information, and decide what part of the notebook to use for prediction.

For:

The movie was not at all good

The LSTM can preserve **not** until it sees **good**.

3. Prerequisites

- RNNs
- Neural networks
- Sigmoid and tanh
- Backpropagation
- Word embeddings

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Cell state	Long-term memory	Preserves information	c_t	LSTM key
Forget gate	Decides what to erase	Removes irrelevant memory	old topic	Formula asked
Input gate	Decides what to write	Adds new info	sentiment word	Gate intuition
Output gate	Decides what to expose	Controls hidden state	prediction state	Hidden vs cell
Bidirectional LSTM	Reads both directions	Better context	NER	Not for causal generation

5. Algorithm / Working Process

1. Convert tokens to embeddings.
2. For each time step, compute gates.
3. Update cell state.
4. Update hidden state.
5. Use final or all hidden states for prediction.
6. Train with task loss.

Input: token embeddings.

Output: sequence states or class logits.

6. Mathematical Foundation

LSTM equations:

```
f_t = sigmoid(W_f [h_{t-1}, x_t] + b_f)
i_t = sigmoid(W_i [h_{t-1}, x_t] + b_i)
g_t = tanh(W_g [h_{t-1}, x_t] + b_g)
c_t = f_t * c_{t-1} + i_t * g_t
o_t = sigmoid(W_o [h_{t-1}, x_t] + b_o)
h_t = o_t * tanh(c_t)
```

Cross-entropy for classification:

```
L = -sum_i y_i log(p_i)
```

7. Practical Implementation

```
import torch
from torch import nn

class TextLSTM(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, num_classes):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True,
                             bidirectional=True)
        self.classifier = nn.Linear(hidden_dim * 2, num_classes)

    def forward(self, input_ids):
        x = self.embedding(input_ids)
        output, (hidden, cell) = self.lstm(x)
        final = torch.cat([hidden[-2], hidden[-1]], dim=1)
        return self.classifier(final)

model = TextLSTM(5000, 100, 128, 2)
batch = torch.randint(1, 5000, (4, 30))
print(model(batch).shape)
```

8. Code Explanation

The embedding layer maps tokens to vectors. The bidirectional LSTM reads text left-to-right and right-to-left. The last forward and backward hidden states are concatenated and passed to the classifier.

9. Training / Evaluation

Use padded batches, train/validation/test split, and cross-entropy loss.

Metrics:

- Classification: accuracy, F1
- Sequence labeling: token-level F1
- Language modeling: perplexity

Improvements:

- Pretrained embeddings
- Dropout
- Gradient clipping
- Attention layer
- Packed padded sequences

10. Complexity and Cost

```
Time:  $O(T * 4(H^2 + HD))$   
Memory:  $O(TH)$ 
```

The factor 4 comes from the four gate computations.

11. Common Use Cases

- Sentiment analysis
- NER
- POS tagging
- Text classification
- Time-series sequence modeling
- Speech/text sequence modeling

12. Common Mistakes

- Forgetting bidirectional doubles hidden dimension
- Not masking padding
- Using BiLSTM for causal generation
- Not clipping exploding gradients
- Confusing hidden state and cell state

13. Edge Cases / Limitations

LSTMs are better than vanilla RNNs but still sequential and slower than transformers on large datasets. Very long documents remain difficult.

14. Variations

Variation	What changes	When to use	Importance
UniLSTM	One direction	Causal tasks	High

Variation	What changes	When to use	Importance
BiLSTM	Two directions	Classification, NER	High
Stacked LSTM	Multiple layers	More capacity	Medium
LSTM with attention	Weighted states	Long texts	Medium
GRU	Fewer gates	Faster alternative	Medium

15. Related Topics

- RNN: LSTM is a gated RNN.
- GRU: simpler gated alternative.
- Attention: improves focus over long sequences.
- Transformer: parallel alternative replacing recurrence.

16. Interview Questions

1. What is an LSTM? Answer: A gated RNN with cell state for long-term memory.
2. Why was LSTM introduced? Answer: To reduce vanishing gradient problems in RNNs.
3. Name the LSTM gates. Answer: Forget, input, and output gates.
4. What does the forget gate do? Answer: Decides how much previous cell state to keep.
5. Hidden state vs cell state? Answer: Cell stores memory; hidden is exposed output.
6. Why use sigmoid in gates? Answer: It outputs values between 0 and 1 for controlling flow.
7. What is BiLSTM? Answer: LSTM that reads sequence in both directions.
8. When should BiLSTM not be used? Answer: Causal generation where future tokens are unavailable.
9. LSTM vs Transformer? Answer: LSTM is sequential; Transformer uses self-attention and parallelization.
10. Why gradient clipping? Answer: To control exploding gradients.

17. Practice Tasks

- Implement BiLSTM sentiment classifier.
- Add dropout to LSTM.
- Compare packed vs padded sequences.
- Visualize hidden states.
- Add attention on top of BiLSTM.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
BiLSTM Sentiment Model	Classifies reviews	PyTorch	IMDb	Classic deep NLP project

Project	What it does	Tech stack	Dataset	Resume value
NER Tagger	Extracts entities	PyTorch	CoNLL-2003	Strong sequence labeling
Toxic Comment Classifier	Detects toxicity	PyTorch	Jigsaw	Real moderation use case

19. Quick Revision

- Key idea: gated memory for sequences.
- Main formula: $c_t = f_t * c_{t-1} + i_t * g_t$.
- When to use: sequence tasks needing order with moderate data.
- Important metrics: F1, accuracy, perplexity.
- Common traps: padding, dimension mismatch, BiLSTM in causal tasks.
- Interview one-liner: LSTM uses gates to remember useful information across time.

20. Final Cheat Sheet

Item	Summary
Definition	Gated RNN with cell memory
Input/output	Token embeddings -> hidden states/logits
Main steps	gate, update cell, output hidden
Key hyperparameters	hidden size, layers, dropout, bidirectional
Metrics	accuracy, F1, perplexity
Pros	Better long memory than RNN
Cons	Sequential, slower than transformers
Best use cases	Classification and sequence labeling

Attention

1. Overview

Attention is a mechanism that lets a model focus on the most relevant parts of an input when making a prediction. Instead of compressing an entire sequence into one fixed vector, attention computes weighted combinations of token representations.

It is central to transformers, machine translation, summarization, question answering, image captioning, and multimodal AI.

2. Intuition

When answering a question, humans focus on relevant words.

Question:

Where did Alice go?

Sentence:

Alice went to Paris after lunch.

Attention should focus strongly on Paris.

3. Prerequisites

- Vectors and dot products
- Softmax
- RNN/LSTM basics
- Matrix multiplication
- Sequence modeling

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Query	What we are looking for	Drives attention	decoder state	Q/K/V roles
Key	What each token offers	Used for matching	encoder token	Dot product score
Value	Information to retrieve	Weighted output	token representation	Weighted sum
Attention score	Relevance measure	Determines focus	QK^T	Scaling factor
Softmax weights	Convert scores to probabilities	Sum to 1	attention distribution	Interpretability
Self-attention	Tokens attend to each other	Transformer core	word-word context	Parallel context

5. Algorithm / Working Process

Scaled dot-product attention:

1. Project input into queries, keys, and values.
2. Compute similarity scores between queries and keys.
3. Scale scores by square root of key dimension.
4. Apply mask if needed.
5. Apply softmax to get weights.
6. Multiply weights by values.

Input: token representations.

Output: context-aware representations.

6. Mathematical Foundation

Attention formula:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

where:

- Q = queries
- K = keys
- V = values
- d_k = key dimension

Softmax:

$$\text{softmax}(z_i) = \exp(z_i) / \sum_j \exp(z_j)$$

The scaling prevents dot products from becoming too large and causing tiny gradients.

7. Practical Implementation

```
import torch
import torch.nn.functional as F

def scaled_dot_product_attention(Q, K, V, mask=None):
    d_k = Q.size(-1)
    scores = Q @ K.transpose(-2, -1) / (d_k ** 0.5)

    if mask is not None:
        scores = scores.masked_fill(mask == 0, float("-inf"))

    weights = F.softmax(scores, dim=-1)
    output = weights @ V
    return output, weights

Q = torch.randn(2, 4, 8)
K = torch.randn(2, 4, 8)
V = torch.randn(2, 4, 8)
output, weights = scaled_dot_product_attention(Q, K, V)
print(output.shape, weights.shape)
```

8. Code Explanation

The function computes attention scores using matrix multiplication, scales by $\sqrt{d_k}$, applies optional masking, converts scores to weights with softmax, and returns the weighted sum of values.

9. Training / Evaluation

Attention is trained end-to-end as part of a neural network. Evaluate by the task:

- Translation: BLEU
- Summarization: ROUGE
- Classification: F1/accuracy
- QA: exact match/F1

Attention weights can be inspected, but they are not always faithful explanations.

10. Complexity and Cost

For sequence length n and hidden size d :

```
Time:  $O(n^2 d)$   
Memory:  $O(n^2)$ 
```

The quadratic sequence cost is a major limitation for long documents.

11. Common Use Cases

- Machine translation
- Summarization
- Question answering
- Transformers
- Image captioning
- Speech recognition
- Vision-language models

12. Common Mistakes

- Forgetting scaling by $\sqrt{d_k}$
- Applying softmax over wrong dimension
- Ignoring masks
- Treating attention as guaranteed explanation
- Confusing self-attention with cross-attention

13. Edge Cases / Limitations

Attention can be expensive for long sequences. It may attend to irrelevant tokens if training data is poor. Attention weights are not always human-interpretable explanations.

14. Variations

Variation	What changes	When to use	Importance
Additive attention	Uses feed-forward score	RNN seq2seq	Medium
Dot-product attention	Uses dot product	Efficient matching	High
Scaled dot-product	Adds scaling	Transformers	High
Multi-head attention	Multiple attention heads	Rich relations	High
Cross-attention	Query from one source, key/value from another	Encoder-decoder, multimodal	High
Causal attention	Masks future tokens	GPT	High

15. Related Topics

- Transformer: built from self-attention.
- BERT: uses bidirectional self-attention.
- GPT: uses causal self-attention.
- Multimodal models: use cross-attention or joint attention.
- RNN seq2seq: attention fixed encoder bottleneck.

16. Interview Questions

1. What is attention? Answer: A mechanism that computes weighted focus over input representations.
2. What are Q, K, and V? Answer: Query searches, key matches, value provides retrieved information.
3. Write the attention formula. Answer: $\text{softmax}(QK^T/\sqrt{d_k})V$.
4. Why scale by $\sqrt{d_k}$? Answer: To prevent large dot products and unstable softmax.
5. What is self-attention? Answer: Tokens attend to tokens in the same sequence.
6. What is cross-attention? Answer: Queries attend to keys/values from another sequence or modality.
7. What is causal masking? Answer: Preventing a token from attending to future tokens.
8. Why is attention expensive? Answer: It computes pairwise token interactions, $O(n^2)$.
9. Does attention explain model decisions? Answer: It can help inspect behavior, but is not always faithful.
10. Why multi-head attention? Answer: Different heads capture different relation patterns.

17. Practice Tasks

- Implement scaled dot-product attention.
- Add causal masking.
- Visualize attention weights.

- Compare single-head and multi-head outputs.
- Use attention on top of LSTM for classification.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Attention Visualizer	Shows token-token attention	PyTorch, Streamlit	Custom text	Great interview demo
LSTM with Attention	Classifies reviews	PyTorch	IMDb	Bridges RNN to transformer
Mini Translator	Uses seq2seq attention	PyTorch	English-French toy data	Classic NLP project

19. Quick Revision

- Key idea: weighted focus over relevant tokens.
- Main formula: $\text{softmax}(QK^T/\sqrt{d_k})V$.
- When to use: sequence and multimodal modeling.
- Important metrics: task-specific.
- Common traps: wrong mask, wrong softmax dimension.
- Interview one-liner: attention lets each token dynamically gather relevant context.

20. Final Cheat Sheet

Item	Summary
Definition	Weighted information retrieval mechanism
Input/output	Q,K,V -> context vectors
Main steps	score, scale, mask, softmax, weighted sum
Key hyperparameters	heads, hidden size, mask
Metrics	task-specific
Pros	Captures dependencies, parallelizable
Cons	Quadratic cost
Best use cases	Transformers, QA, translation, multimodal AI

Transformer

1. Overview

The Transformer is a neural architecture based on self-attention, introduced to avoid recurrence and enable parallel sequence processing. It is the foundation of BERT, GPT, T5, modern LLMs, and many vision-language

models.

Transformers are used in translation, summarization, chatbots, code generation, retrieval, classification, speech, vision, and multimodal AI.

2. Intuition

Instead of reading words one by one like an RNN, a transformer lets every token look at every other token directly.

In:

```
The animal did not cross the street because it was tired.
```

Self-attention helps connect **it** to **animal**.

3. Prerequisites

- Attention
- Matrix multiplication
- Neural networks
- Embeddings
- Layer normalization
- Residual connections
- Cross-entropy

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Token embedding	Converts IDs to vectors	Model input	$E[id]$	Learned embeddings
Positional encoding	Adds order information	Attention alone has no order	position vector	Sinusoidal vs learned
Multi-head attention	Parallel attention heads	Captures diverse relations	syntax and coreference	Formula
Feed-forward network	Per-token MLP	Adds nonlinearity	two linear layers	Same applied to each token
Residual connection	Adds input to output	Stabilizes deep training	$x + layer(x)$	Essential
LayerNorm	Normalizes activations	Stable training	normalize hidden dim	BERT/GPT use
Encoder	Bidirectional context	Understanding tasks	BERT	Encoder-only

Subtopic	Meaning	Why it matters	Example	Interview angle
Decoder	Causal generation	Next-token prediction	GPT	Decoder-only

5. Algorithm / Working Process

Encoder block:

1. Add token and positional embeddings.
2. Apply multi-head self-attention.
3. Add residual connection and layer norm.
4. Apply feed-forward network.
5. Add residual connection and layer norm.
6. Repeat for multiple layers.

Decoder block adds causal masking and may include cross-attention in encoder-decoder models.

Input: token IDs.

Output: contextual token representations or next-token logits.

Training: usually cross-entropy for language modeling or task loss for fine-tuning.

Inference: encoder models output representations; decoder models generate tokens autoregressively.

6. Mathematical Foundation

Self-attention:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$$

Multi-head:

$$\begin{aligned} \text{head}_i &= \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \\ \text{MultiHead} &= \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \end{aligned}$$

Feed-forward:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Language modeling loss:

$$L = -\sum_t \log P(x_t \mid x_{<t})$$

7. Practical Implementation

```
import torch
from torch import nn

encoder_layer = nn.TransformerEncoderLayer(
    d_model=128,
    nhead=4,
    dim_feedforward=256,
    batch_first=True,
)
encoder = nn.TransformerEncoder(encoder_layer, num_layers=2)

tokens = torch.randn(8, 20, 128) # batch, sequence, hidden
output = encoder(tokens)
print(output.shape)
```

8. Code Explanation

`TransformerEncoderLayer` creates one encoder block with self-attention and feed-forward layers.

`TransformerEncoder` stacks blocks. The input is already embedded; real NLP models add token and positional embeddings before this step.

9. Training / Evaluation

Training requires large data for pretraining, but fine-tuning can work with smaller datasets.

Metrics:

- Language modeling: perplexity
- Classification: accuracy/F1
- Translation: BLEU
- Summarization: ROUGE
- QA: exact match/F1

Improvements:

- Better tokenization
- Larger data
- Regularization
- Learning rate scheduling
- Pretraining then fine-tuning

10. Complexity and Cost

Self-attention:

Time: $O(n^2 d)$
Memory: $O(n^2)$

Feed-forward:

Time: $O(n d d_{ff})$

Transformers train efficiently on GPUs/TPUs due to parallelism, but long contexts are expensive.

11. Common Use Cases

- LLMs
- Translation
- Summarization
- Code generation
- Classification
- Search embeddings
- Vision transformers
- Multimodal models

12. Common Mistakes

- Forgetting positional information
- Confusing encoder and decoder models
- Ignoring attention masks
- Underestimating quadratic cost
- Fine-tuning with too high learning rate
- Using random transformers on tiny datasets

13. Edge Cases / Limitations

Transformers need significant data and compute. Long sequences are expensive. They can hallucinate in generation and may inherit training data bias.

14. Variations

Variation	What changes	When to use	Importance
Encoder-only	Bidirectional attention	Classification, NER	High
Decoder-only	Causal attention	Generation	High
Encoder-decoder	Encoder + decoder	Translation, summarization	High
Long-context transformers	Efficient attention	Long documents	Medium
Vision Transformer	Image patches as tokens	Computer vision	Medium

15. Related Topics

- Attention: transformer core operation.
- BERT: encoder-only transformer.
- GPT: decoder-only transformer.
- RNN/LSTM: older sequential models.
- Multimodal VLMs: transformers over text and image tokens.

16. Interview Questions

1. What is a transformer? Answer: A self-attention-based architecture for sequence modeling.
2. Why are transformers parallelizable? Answer: They process all tokens simultaneously using attention.
3. Why positional encoding? Answer: Attention alone has no sequence order.
4. Encoder vs decoder? Answer: Encoder is bidirectional; decoder is causal for generation.
5. What is multi-head attention? Answer: Multiple attention operations in parallel.
6. Why residual connections? Answer: They stabilize deep network training.
7. What is LayerNorm? Answer: Normalization across hidden features.
8. Complexity of self-attention? Answer: $O(n^2 d)$ time and $O(n^2)$ memory.
9. Why did transformers replace RNNs? Answer: Better parallelism and long-range dependency modeling.
10. What is causal masking? Answer: Masking future tokens during autoregressive generation.

17. Practice Tasks

- Implement attention from scratch.
- Train a tiny transformer classifier.
- Compare RNN vs transformer on same dataset.
- Inspect attention masks.
- Fine-tune a pretrained transformer.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Tiny Transformer Classifier	Classifies text	PyTorch	AG News	Strong architecture knowledge
Transformer Explainer	Visualizes attention	PyTorch, Streamlit	Custom text	Interview demo
Mini Language Model	Predicts next token	PyTorch	Tiny Shakespeare	LLM foundation project

19. Quick Revision

- Key idea: self-attention replaces recurrence.
- Main formula: $\text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$.
- When to use: modern NLP and multimodal tasks.
- Important metrics: perplexity, F1, BLEU, ROUGE.
- Common traps: masks, positional encoding, cost.
- Interview one-liner: transformers let every token attend to every other token in parallel.

20. Final Cheat Sheet

Item	Summary
Definition	Self-attention neural architecture
Input/output	Token IDs -> contextual states/logits
Main steps	embed, attend, FFN, residual, norm
Key hyperparameters	layers, heads, hidden size, context length
Metrics	task-specific
Pros	Parallel, powerful, contextual
Cons	Compute-heavy, quadratic attention
Best use cases	LLMs, NLP, vision-language AI

BERT

1. Overview

BERT, Bidirectional Encoder Representations from Transformers, is an encoder-only transformer pretrained to understand text bidirectionally. It learns contextual embeddings by looking at both left and right context.

BERT is widely used for classification, named entity recognition, question answering, semantic similarity, and retrieval embeddings.

2. Intuition

BERT is like a reader that can see the full sentence before answering. If a word is hidden, BERT predicts it using both sides.

The capital of France is [MASK].

BERT predicts **Paris**.

3. Prerequisites

- Transformer encoder

- Self-attention
- Tokenization
- Masked language modeling
- Fine-tuning
- Cross-entropy

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Encoder-only	Uses bidirectional attention	Best for understanding	classification	BERT vs GPT
MLM	Predict masked tokens	Pretraining task	[MASK]	Core BERT objective
NSP	Next sentence prediction	Original sentence-pair task	sentence A/B	Less used now
[CLS] token	Aggregate representation	Classification head	first token	Common fine-tuning
[SEP] token	Separates sentences	Pair inputs	QA/NLI	Segment embeddings
Fine-tuning	Train on labeled task	Adapt BERT	sentiment	Low learning rate

5. Algorithm / Working Process

Pretraining:

1. Tokenize text using WordPiece.
2. Randomly mask some tokens.
3. Feed sequence to transformer encoder.
4. Predict masked tokens.
5. Optimize MLM loss.

Fine-tuning for classification:

1. Add classifier on [CLS].
2. Train on labeled data.
3. Use softmax for prediction.

Input: token IDs, attention mask, token type IDs.

Output: contextual embeddings or task logits.

6. Mathematical Foundation

Masked language modeling:

$$L_{\text{MLM}} = - \sum_{i \in M} \log P(x_i \mid x_{\text{without_masked_tokens}})$$

Classification:

```
h_cls = BERT(input)[CLS]
p = softmax(W h_cls + b)
L = -sum_c y_c log(p_c)
```

BERT uses bidirectional self-attention:

token_i can attend to tokens before and after it

7. Practical Implementation

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch

model_name = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(model_name,
num_labels=2)

text = "This interview guide is useful."
inputs = tokenizer(text, return_tensors="pt", truncation=True, padding=True)

with torch.no_grad():
    logits = model(**inputs).logits
    probs = torch.softmax(logits, dim=-1)

print(probs)
```

8. Code Explanation

The tokenizer converts text into BERT-compatible IDs and masks. `AutoModelForSequenceClassification` loads BERT with a classification head. `softmax` converts logits into class probabilities.

9. Training / Evaluation

Fine-tuning tips:

- Use small learning rates like `2e-5` to `5e-5`.
- Use train/validation/test split.
- Use early stopping.
- Monitor overfitting.

Metrics:

- Classification: accuracy, F1
- NER: entity-level F1
- QA: exact match, F1
- Retrieval: MRR, recall@k

10. Complexity and Cost

BERT-base has about 110M parameters.

Self-attention:

$$O(n^2 \cdot d)$$

Fine-tuning usually needs GPU for speed, though small batches can run on CPU slowly.

11. Common Use Cases

- Sentiment analysis
- NER
- Question answering
- Semantic similarity
- Text classification
- Reranking
- Embedding generation

12. Common Mistakes

- Using BERT for left-to-right generation
- Forgetting attention masks
- Too high learning rate
- Preprocessing away important tokens
- Using [CLS] blindly for semantic similarity without checking
- Ignoring max length truncation

13. Edge Cases / Limitations

BERT is not naturally generative. It has fixed context length and can be expensive. It may not perform well on domains far from pretraining unless adapted.

14. Variations

Variation	What changes	When to use	Importance
RoBERTa	Improved pretraining, no NSP	Strong BERT-like model	High
DistilBERT	Smaller distilled model	Low latency	High

Variation	What changes	When to use	Importance
ALBERT	Parameter sharing	Efficient training	Medium
Sentence-BERT	Better sentence embeddings	Semantic search	High
DomainBERT	Domain pretraining	Bio/legal/finance	Medium

15. Related Topics

- Transformer encoder: BERT architecture.
- GPT: decoder-only generative model.
- WordPiece tokenization: BERT tokenizer.
- Fine-tuning vs feature extraction: two ways to use BERT.
- RAG: BERT-like encoders can be retrievers/rerankers.

16. Interview Questions

1. What is BERT? Answer: A bidirectional encoder-only transformer pretrained for language understanding.
2. What is MLM? Answer: Masked language modeling, predicting hidden tokens from context.
3. BERT vs GPT? Answer: BERT is bidirectional encoder for understanding; GPT is causal decoder for generation.
4. What is **[CLS]** used for? Answer: Classification representation.
5. What is **[SEP]**? Answer: Separator token for sentence boundaries or sentence pairs.
6. Can BERT generate text? Answer: Not naturally autoregressively.
7. Why small learning rate in fine-tuning? Answer: To avoid destroying pretrained weights.
8. What is attention mask? Answer: It marks real tokens vs padding.
9. What is token type ID? Answer: Segment ID used for sentence-pair tasks.
10. Why is BERT contextual? Answer: Word representation depends on surrounding tokens.

17. Practice Tasks

- Fine-tune BERT for sentiment classification.
- Extract embeddings from BERT.
- Compare BERT and TF-IDF on AG News.
- Fine-tune DistilBERT for low latency.
- Analyze truncation errors on long documents.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
BERT Sentiment Analyzer	Fine-tunes BERT	HF, PyTorch	IMDb	Strong placement project
NER Resume Parser	Extracts skills/entities	HF, spaCy	Resume data	AI engineer relevance
Semantic FAQ Matcher	Finds similar questions	Sentence-BERT	Quora pairs	RAG/search relevance

19. Quick Revision

- Key idea: bidirectional transformer encoder.
- Main formula: MLM loss over masked positions.
- When to use: language understanding tasks.
- Important metrics: F1, accuracy, EM.
- Common traps: using for generation, wrong tokenizer.
- Interview one-liner: BERT learns contextual meaning by predicting masked words using both-side context.

20. Final Cheat Sheet

Item	Summary
Definition	Bidirectional encoder transformer
Input/output	Text IDs -> contextual states/task logits
Main steps	tokenize, encode, classify/extract
Key hyperparameters	learning rate, max length, batch size
Metrics	F1, accuracy, EM
Pros	Strong understanding model
Cons	Not causal generation, costly
Best use cases	Classification, NER, QA, reranking

GPT-style Models

1. Overview

GPT-style models are decoder-only transformers trained with causal language modeling: predicting the next token given previous tokens. They are the foundation of modern chatbots, code assistants, summarizers, agents, and generative AI systems.

2. Intuition

GPT learns to continue text.

Input: The capital of Japan is
Output: Tokyo

By learning next-token prediction at massive scale, it develops broad language, reasoning, and instruction-following abilities.

3. Prerequisites

- Transformer decoder
- Causal self-attention
- Tokenization
- Cross-entropy
- Sampling
- Fine-tuning and alignment basics

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Decoder-only transformer	Uses causal blocks	Generation	GPT	BERT vs GPT
Causal mask	Blocks future tokens	Prevents cheating	token sees left context	Essential
Next-token prediction	Training objective	Self-supervised learning	predict Tokyo	Formula
Prompting	Conditioning with input text	Controls output	instruction prompt	Prompt engineering
Sampling	Choose next token	Controls creativity	temperature	Decoding methods
Alignment	Make model helpful/safe	Chat behavior	instruction tuning	RLHF/DPO
Context window	Max tokens model sees	Long-doc limits	8k/128k tokens	Cost and memory

5. Algorithm / Working Process

Training:

1. Tokenize text corpus.
2. Create sequences.
3. Use causal mask.

4. Predict each next token.
5. Optimize cross-entropy.

Inference:

1. Tokenize prompt.
2. Run model to get next-token logits.
3. Convert logits to probabilities.
4. Select token using greedy/sampling/beam.
5. Append token and repeat.

Input: prompt tokens.

Output: generated tokens.

6. Mathematical Foundation

Autoregressive factorization:

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_{<t})$$

Training loss:

$$L = -\sum_{t=1}^T \log P(x_t \mid x_{<t})$$

Softmax:

$$P(\text{token}_i) = \exp(\text{logit}_i / \text{temperature}) / \sum_j \exp(\text{logit}_j / \text{temperature})$$

Lower temperature makes output more deterministic; higher temperature increases randomness.

7. Practical Implementation

```
from transformers import AutoTokenizer, AutoModelForCausalLM

model_name = "distilgpt2"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name)

prompt = "Natural language processing is useful because"
inputs = tokenizer(prompt, return_tensors="pt")

output_ids = model.generate(
    **inputs,
    max_new_tokens=40,
```

```
do_sample=True,  
temperature=0.8,  
top_p=0.9,  
)  
  
print(tokenizer.decode(output_ids[0], skip_special_tokens=True))
```

8. Code Explanation

`AutoModelForCausalLM` loads a decoder-style language model. `generate` repeatedly predicts and appends tokens. `temperature` and `top_p` control randomness and diversity.

9. Training / Evaluation

Training stages:

- Pretraining on large corpus
- Supervised instruction fine-tuning
- Preference tuning such as RLHF or DPO
- Task-specific fine-tuning or prompting

Metrics:

- Perplexity
- Exact match/F1 for QA
- BLEU/ROUGE for generation tasks
- Human evaluation
- Safety and hallucination rate

10. Complexity and Cost

Training large GPT models is extremely expensive.

Attention cost:

$O(n^2 \cdot d)$

Autoregressive inference generates one token at a time, but KV caching reduces repeated computation.

Memory depends on:

- Parameters
- Activations
- Context length
- KV cache

11. Common Use Cases

- Chatbots

- Code generation
- Summarization
- Content generation
- Agents
- Data extraction
- RAG answer generation
- Synthetic data

12. Common Mistakes

- Assuming generated text is always factual
- Ignoring prompt injection
- Using high temperature for factual tasks
- Not validating outputs
- Confusing pretraining, fine-tuning, and prompting
- Sending too much irrelevant context

13. Edge Cases / Limitations

GPT-style models can hallucinate, be sensitive to prompts, struggle with exact arithmetic, and reflect training data biases. Context windows are finite, and inference cost grows with generated tokens.

14. Variations

Variation	What changes	When to use	Importance
Base LM	Predicts next token	Research/pretraining	Medium
Instruction-tuned LM	Follows instructions	Chat and assistants	High
Code LM	Trained heavily on code	Coding tasks	High
MoE LM	Sparse expert routing	Scale efficiently	Medium
Multimodal GPT	Adds image/audio/video inputs	Vision-language tasks	High

15. Related Topics

- Transformer decoder: GPT architecture.
- BERT: bidirectional encoder model.
- RAG: grounds GPT outputs in retrieved documents.
- LoRA/QLoRA: efficient fine-tuning.
- Attention: core computation.

16. Interview Questions

1. What is a GPT-style model? Answer: A decoder-only transformer trained to predict the next token.
2. What is causal masking? Answer: Preventing tokens from seeing future tokens.
3. GPT vs BERT? Answer: GPT is generative and causal; BERT is bidirectional and understanding-focused.

4. What is autoregressive generation? Answer: Generating one token at a time conditioned on previous tokens.
5. Write the GPT loss. Answer: $L = -\sum_t \log P(x_t | x_{<t})$.
6. What is temperature? Answer: A sampling parameter controlling randomness.
7. What is top-p sampling? Answer: Sampling from the smallest token set whose cumulative probability exceeds p .
8. What is hallucination? Answer: Confident generation of unsupported or false information.
9. What is KV cache? Answer: Stored attention keys/values used to speed autoregressive inference.
10. What is instruction tuning? Answer: Fine-tuning a base model to follow human instructions.

17. Practice Tasks

- Generate text using a small causal LM.
- Compare greedy, temperature, and top-p decoding.
- Measure token count and latency.
- Build a RAG chatbot with a small LM.
- Evaluate hallucinations on factual prompts.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Mini GPT Text Generator	Generates domain text	HF, PyTorch	Tiny Shakespeare/domain text	LLM foundation
RAG Study Assistant	Answers from documents	HF/OpenAI, FAISS	PDFs/notes	AI engineer relevance
Prompt Evaluation Harness	Tests prompts systematically	Python, pandas	Custom prompts	Strong production skill

19. Quick Revision

- Key idea: predict next token using previous tokens.
- Main formula: $P(x) = \text{product } P(x_t | x_{<t})$.
- When to use: generation, chat, summarization, agents.
- Important metrics: perplexity, task score, human eval.
- Common traps: hallucination, prompt injection, cost.
- Interview one-liner: GPT-style models are causal transformers optimized for next-token generation.

20. Final Cheat Sheet

Item	Summary
Definition	Decoder-only autoregressive transformer
Input/output	Prompt -> generated tokens
Main steps	tokenize, causal attend, predict, sample
Key hyperparameters	temperature, top_p, max tokens, context length
Metrics	perplexity, F1, human eval
Pros	Flexible generation and reasoning
Cons	Hallucination, cost, prompt sensitivity
Best use cases	Chat, code, summarization, RAG

Multimodal Vision-Language Models

1. Overview

Multimodal vision-language models, or VLMs, process both visual inputs and text. They connect image understanding with language generation or reasoning.

They are used in image captioning, visual question answering, document AI, OCR reasoning, medical imaging assistants, robotics, autonomous driving, UI understanding, and multimodal chatbots.

2. Intuition

A VLM combines the eyes of a vision model with the language ability of an LLM.

Input:

```
Image: a chart showing sales rising
Question: What trend is shown?
```

Output:

```
Sales increase over time.
```

3. Prerequisites

- CNNs or Vision Transformers
- Transformers
- Attention and cross-attention
- Image preprocessing
- Tokenization

- Contrastive learning
- LLM basics

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Vision encoder	Converts image to visual features	Model sees image	ViT/CLIP encoder	Patch embeddings
Text encoder/decoder	Processes language	Understands or generates text	BERT/GPT	Encoder vs decoder
Image patches	Image split into tokens	Transformer-friendly	16x16 patches	ViT connection
Contrastive learning	Aligns image and text embeddings	CLIP-style matching	caption-image pairs	InfoNCE loss
Cross-attention	Text attends to image features	Fusion mechanism	question attends image	Multimodal reasoning
Projection layer	Maps vision features to LLM space	Connects modalities	linear adapter	Common architecture
Instruction tuning	Teaches multimodal responses	Chat behavior	image QA	Modern VLMs

5. Algorithm / Working Process

Common VLM pipeline:

1. Preprocess image into patches or pixels.
2. Vision encoder converts image to visual embeddings.
3. Tokenizer converts prompt to text tokens.
4. Fusion module aligns image and text features.
5. Language model generates or classifies output.

Training options:

- Contrastive image-text training
- Caption generation
- Visual question answering
- Multimodal instruction tuning

Inference:

1. User provides image and prompt.
2. Model encodes image and text.
3. Model generates answer token by token.

6. Mathematical Foundation

CLIP-style contrastive loss:

For image embedding i and text embedding t :

$$s(i, t) = \text{cosine}(i, t) / \tau$$

Image-to-text loss:

$$L_i = -\log \exp(s(i_i, t_i)) / \sum_j \exp(s(i_i, t_j))$$

Text-to-image loss:

$$L_t = -\log \exp(s(t_i, i_i)) / \sum_j \exp(s(t_i, i_j))$$

Total:

$$L = (L_i + L_t) / 2$$

For generative VLMs:

$$L = -\sum_t \log P(y_t \mid y_{<t}, \text{image}, \text{prompt})$$

7. Practical Implementation

```
from PIL import Image
import requests
from transformers import CLIPProcessor, CLIPModel

model_name = "openai/clip-vit-base-patch32"
model = CLIPModel.from_pretrained(model_name)
processor = CLIPProcessor.from_pretrained(model_name)

image = Image.open(
    requests.get(
        "https://huggingface.co/datasets/huggingface/documentation-
images/resolve/main/cats.png",
        stream=True,
    ).raw
)

texts = ["a photo of a cat", "a photo of a dog", "a chart"]
inputs = processor(text=texts, images=image, return_tensors="pt", padding=True)
```

```
outputs = model(**inputs)

probs = outputs.logits_per_image.softmax(dim=1)
print(dict(zip(texts, probs[0].tolist())))
```

8. Code Explanation

CLIP encodes the image and candidate text descriptions into a shared embedding space. `logits_per_image` gives similarity scores between the image and each text. Softmax converts these scores into probabilities over candidate captions.

9. Training / Evaluation

Datasets:

- Image-caption pairs
- VQA datasets
- OCR/document datasets
- Multimodal instruction datasets

Metrics:

- Retrieval: recall@k
- Captioning: BLEU, CIDEr, SPICE
- VQA: accuracy
- OCR/document QA: exact match/F1
- Safety: hallucination and refusal quality

Improvements:

- Better image resolution
- More diverse multimodal data
- Stronger LLM backbone
- Better OCR integration
- Fine-tuning with task-specific data

10. Complexity and Cost

Cost depends on:

- Image resolution
- Number of image patches
- Vision encoder size
- LLM size
- Output token length

For ViT attention over image patches:

Time: $O(p^2 d)$

where p is number of patches.

Generative VLMs also pay autoregressive decoding cost.

11. Common Use Cases

- Image captioning
- Visual question answering
- OCR and document understanding
- Medical image assistance
- Chart interpretation
- UI automation
- Robotics perception-language control
- Product search by image

12. Common Mistakes

- Assuming VLMs read tiny text reliably
- Ignoring image resolution
- Treating visual answers as always factual
- Not checking hallucinations
- Evaluating only with language metrics
- Forgetting visual grounding
- Using CLIP for generation even though it is retrieval-style

13. Edge Cases / Limitations

VLMs may fail on:

- Small text in images
- Medical/legal high-stakes interpretation
- Spatial reasoning
- Counting objects
- Fine-grained visual differences
- Unusual diagrams
- Adversarial images

14. Variations

Variation	What changes	When to use	Importance
CLIP-style models	Contrastive image-text matching	Retrieval, zero-shot classification	High
Captioning models	Generate captions	Image description	Medium

Variation	What changes	When to use	Importance
VQA models	Answer visual questions	QA over images	High
LLaVA-style models	Vision encoder + LLM	Multimodal chat	High
Document VLMs	OCR/layout-aware	PDFs, forms, invoices	High
Video-language models	Add temporal frames	Video QA/captioning	Medium

15. Related Topics

- Vision Transformer: image patches as tokens.
- CLIP: contrastive image-text alignment.
- GPT-style models: language decoder for multimodal generation.
- Cross-attention: connects image and text features.
- RAG: multimodal RAG retrieves text/images before answering.

16. Interview Questions

1. What is a vision-language model? Answer: A model that processes both images and text.
2. What is CLIP? Answer: A contrastive model aligning image and text embeddings.
3. How does CLIP train? Answer: It pulls matching image-text pairs together and pushes mismatches apart.
4. What is visual question answering? Answer: Answering natural language questions about an image.
5. How does a generative VLM connect images to an LLM? Answer: A vision encoder produces visual embeddings projected into the LLM input space.
6. What is cross-attention used for? Answer: Letting text tokens attend to image features.
7. Why does image resolution matter? Answer: Small details and text may be lost at low resolution.
8. What is a common VLM limitation? Answer: Hallucinating visual details or failing at counting/spatial reasoning.
9. CLIP vs LLaVA-style model? Answer: CLIP is mainly embedding/retrieval; LLaVA-style models generate language answers.
10. How do you evaluate VLMs? Answer: Use task metrics like recall@k, VQA accuracy, caption metrics, and human evaluation.

17. Practice Tasks

- Use CLIP for zero-shot image classification.
- Build image-text retrieval with embeddings.
- Test a VLM on chart images and record failures.
- Compare OCR + LLM vs VLM on document QA.
- Fine-tune a small VQA model on a narrow dataset.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Image Search with Text	Retrieves images using text queries	CLIP, FAISS, Streamlit	Flickr8k/MS-COCO	Strong multimodal project
Chart QA Assistant	Answers questions about charts	VLM/OCR, Python	ChartQA	AI engineer relevance
Document Visual QA	Answers from scanned forms	OCR, VLM, HF	DocVQA	High resume value

19. Quick Revision

- Key idea: align visual and language representations.
- Main formula: contrastive loss or conditional generation loss.
- When to use: tasks needing image plus text understanding.
- Important metrics: recall@k, VQA accuracy, EM/F1.
- Common traps: hallucination, tiny text, spatial/counting errors.
- Interview one-liner: VLMs connect vision encoders with language models to reason over images and text.

20. Final Cheat Sheet

Item	Summary
Definition	Model that processes images and language
Input/output	Image + text -> label, retrieval result, or generated answer
Main steps	encode image, encode text, fuse, predict/generate
Key hyperparameters	image size, patch size, embedding dim, LLM size
Metrics	recall@k, VQA accuracy, CIDEr, F1
Pros	Enables visual reasoning and multimodal chat
Cons	Costly, hallucination, weak small-text/counting
Best use cases	Image QA, document AI, visual search, multimodal assistants

Combined Interview Questions Across All Topics

1. Why is preprocessing more important for BoW than for BERT? Answer: BoW directly depends on surface tokens, while BERT uses a learned tokenizer and contextual modeling.
2. BoW vs TF-IDF? Answer: BoW counts words; TF-IDF weights words by document-specific importance and corpus rarity.

3. Stemming vs lemmatization? Answer: Stemming is crude suffix stripping; lemmatization returns dictionary forms using linguistic information.
4. Word2Vec vs GloVe? Answer: Word2Vec is predictive and local-context based; GloVe uses global co-occurrence statistics.
5. Static vs contextual embeddings? Answer: Static embeddings give one vector per word; contextual embeddings vary by sentence.
6. RNN vs LSTM? Answer: LSTM adds gates and cell state to preserve long-term information.
7. LSTM vs Transformer? Answer: LSTM is sequential; Transformer uses parallel self-attention.
8. BERT vs GPT? Answer: BERT is bidirectional encoder for understanding; GPT is causal decoder for generation.
9. What is attention? Answer: A weighted retrieval mechanism over token representations.
10. Why are transformers expensive for long inputs? Answer: Self-attention computes pairwise token interactions, giving quadratic cost.
11. Why do GPT models hallucinate? Answer: They optimize likely next-token generation, not guaranteed factual correctness.
12. How do VLMs connect images and text? Answer: Through shared embedding spaces, projection layers, cross-attention, or multimodal token fusion.

Final Roadmap for Placement Preparation

Level	Topics to master	Goal
Beginner	preprocessing, tokenization, stemming, lemmatization, stopwords	Clean and prepare text correctly
Classical NLP	BoW, TF-IDF, Logistic Regression, Naive Bayes	Build strong baselines
Neural NLP	embeddings, Word2Vec, GloVe, RNN, LSTM	Understand sequence modeling
Modern NLP	attention, transformers, BERT, GPT	Explain current NLP systems
Advanced AI	multimodal VLMs, RAG, fine-tuning, evaluation	Build production-style AI projects

Best interview strategy:

1. Start with a simple definition.
2. Give intuition with an example.
3. Mention the formula if relevant.

4. Explain one practical use case.
5. Discuss limitations and common mistakes.
6. Compare with related methods.